Kubernetes Hands-On Lab — Lab Report

Kubernetes, Hands-On (60-minute lab) · k8slabs.learnwithdutta.com

Name	Zeel Dadhaniya
Date performed	23 October 2026 (12:27 – 12:39 UTC)
Scope	Setup, Steps 1–6, the command reference and Cleanup (Options A and B), plus a short volumes and persistent-storage exercise. The final "Container to Kubernetes: Hands-On Assignment" is not part of this report.
Environment	Ubuntu 24.04 Linux VM (4 vCPU, 15 GiB RAM) · Docker Engine 29.9.1 · kind v0.30.0 · kubectl v1.34.0 · Kubernetes v1.34.0 (kindest/node) · Chromium for the browser screenshots
Cluster	k8s-lab: 1 control plane + 2 workers, host ports 30000–30050 mapped
Files produced	kind-config.yaml, deployment.yaml, service.yaml, storage.yaml (see Appendix B)

Contents

1. Notes on the environment
2. Setup — building the cluster
3. Step 1 — Look at your cluster
4. Step 2 — Run your first app
5. Step 3 — Open it in a browser
6. Step 4 — Scale it, then kill it
7. Step 5 — Update, redeploy, undo
8. Step 6 — Break it and fix it
9. Extra — Useful commands, volumes and persistent storage
10. Cleanup
11. What I learned
12. Check yourself: answers
13. Appendix A: all commands in order
14. Appendix B: YAML files

Every screenshot was taken while the command ran, in the order shown. Each caption gives the capture time (UTC). Terminal screenshots show the real terminal contents, and browser screenshots show the real pages served by the cluster.

Notes on the environment

The lab was done on a Linux VM, not WSL on a laptop. Two extra setup steps were needed, and they cause two small differences from the sample outputs in the manual. Everything else follows the manual exactly.

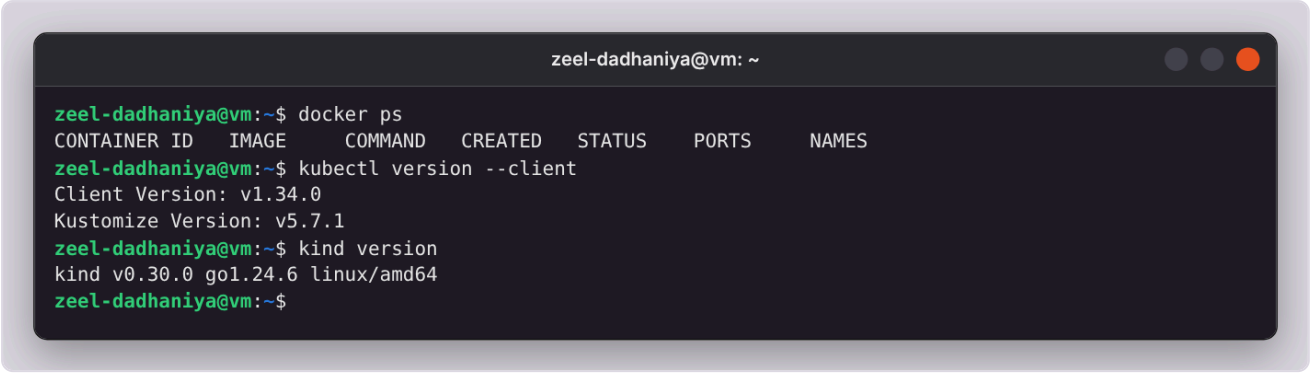
1. **containerd setting in kind-config.yaml** (Figure 4). On this VM a container may not lower its OOM score. The system Pods (etcd, kube-apiserver and others) request a score of -998, so the first `kind create cluster` attempt failed with `failed to update /proc/self/oom_score_adj: Permission denied`. The setting `restrict_oom_score_adj = true` makes containerd limit the value instead of failing. It does not change anything the lab teaches.
2. **Images loaded into the nodes** (Figure 7). The kind nodes on this VM have no direct route to Docker Hub, so the three images (nginx:stable, nginx:stable-alpine, burgxy/webapps:v4) were pulled by the host's Docker and copied in with `kind load image-archive`. The YAML files and image names are the same as in the manual.
3. **Effect on Step 6.** Because the nodes cannot reach the registry, the describe Events for the broken Pod show a connection error instead of "not found". The cause was confirmed separately: Docker Hub returns 404 for that tag (Figure 42).
4. **Step 5 outputs.** The manual makes two updates (stable-alpine, then webapps:v4), so `rollout undo` returned to the previous revision, nginx:stable-alpine. The manual's sample says nginx:stable. Applying the file afterwards first printed `configured` (it restored the file's webapps:v4) and then `unchanged` on the second apply. Both outputs are explained at Figures 34–35.

Setup — building the cluster

Check the tools, create a working folder, describe a 3-node cluster in a kind config file and build it.

Figure 1. Checking that Docker, kubectl and kind are installed

Command	Purpose
`docker ps`	Confirms the Docker daemon is running (an empty table means no containers yet).
`kubectl version --client`	Prints the version of the Kubernetes command-line client.
`kind version`	Prints the version of kind (Kubernetes IN Docker), used to build the cluster.

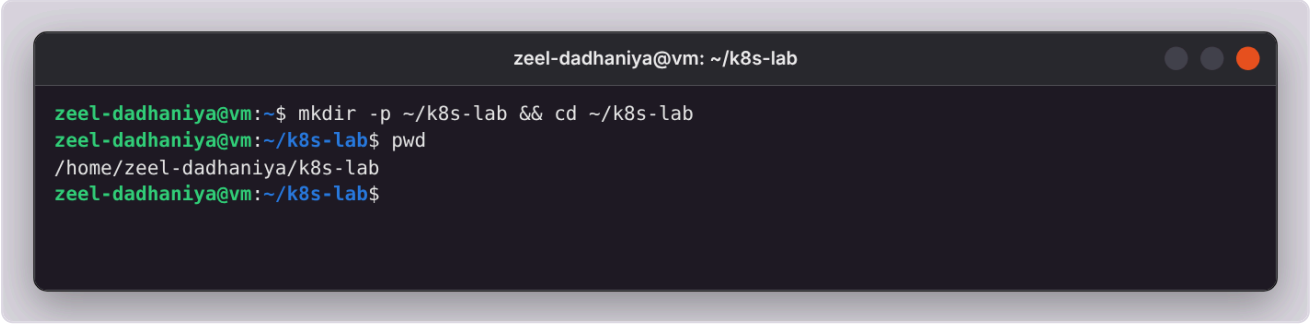


Terminal 1 · captured 2026-10-23 12:27:01 UTC

> **Observation:** All three tools respond: Docker is running with no containers, kubectl is v1.34.0 and kind is v0.30.0.

Figure 2. Creating the lab folder and moving into it

Command	Purpose
`mkdir -p ~/k8s-lab && cd ~/k8s-lab`	Creates the lab folder (if it does not exist) and enters it. All lab files live here.
`pwd`	Prints the current directory to confirm the location.

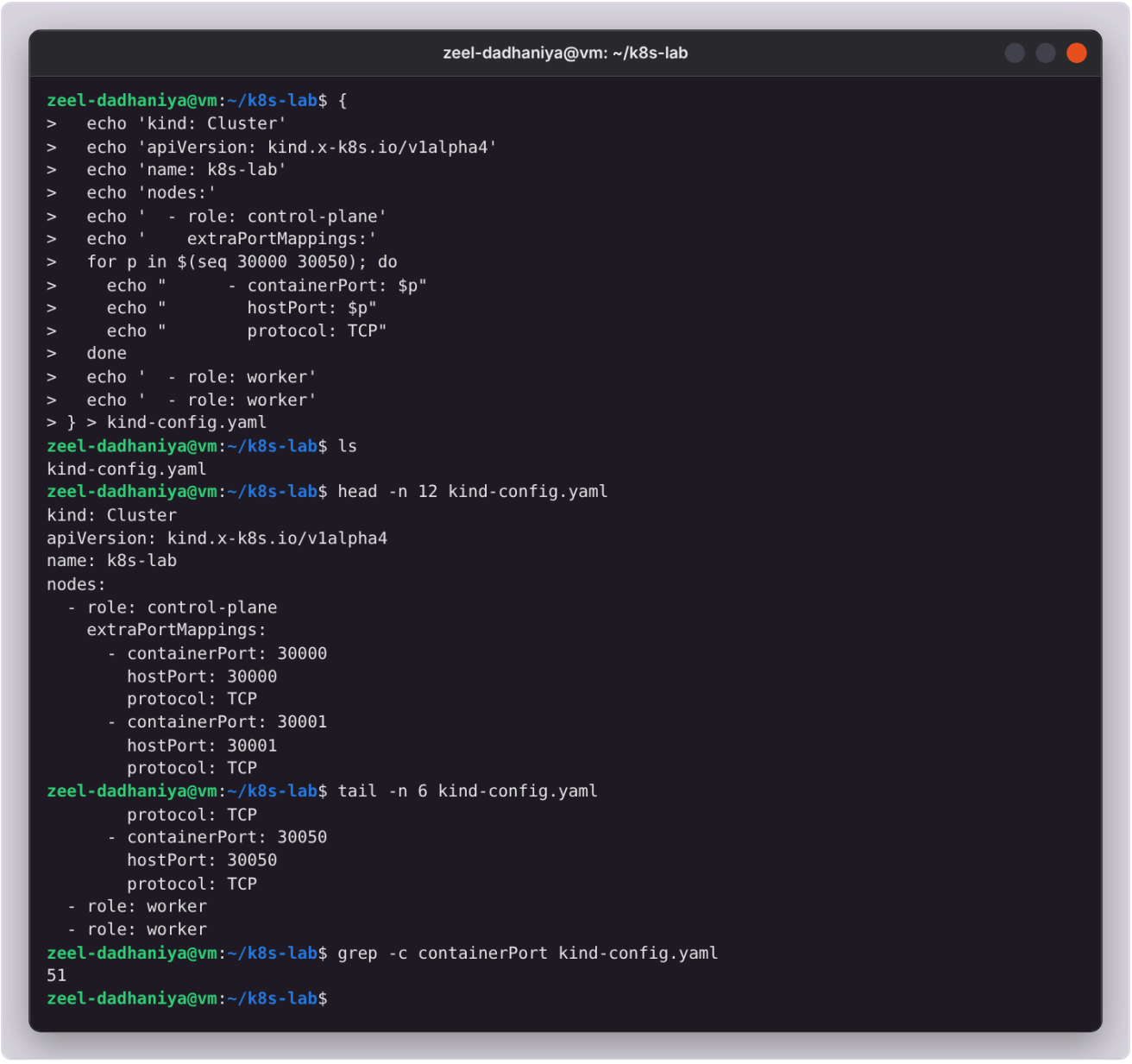


Terminal 1 · captured 2026-10-23 12:27:03 UTC

> **Observation:** The working directory is ~/k8s-lab.

Figure 3. Generating kind-config.yaml with the lab's script

Command	Purpose
`{ echo ...; for p in $(seq 30000 30050); do ... done; ... } > kind-config.yaml`	Writes the cluster description: one control-plane node with ports 30000–30050 mapped to the host, plus two worker nodes.
`ls / head -n 12 / tail -n 6 kind-config.yaml`	Shows the beginning and end of the generated file.
`grep -c containerPort kind-config.yaml`	Counts the port mappings (51 ports: 30000 to 30050).



Terminal 1 · captured 2026-10-23 12:27:14 UTC

> **Observation:** The file describes 1 control plane and 2 workers, and opens 51 NodePorts so the browser can reach apps later.

Figure 4. Environment fix: one containerd setting for this VM

Command	Purpose
`cat >> kind-config.yaml <<'EOF' ... restrict_oom_score_adj = true ... EOF`	Appends a containerdConfigPatches block. On this VM, containers may not lower their OOM score, and system Pods request -998. With this setting containerd limits the value instead of failing.
`tail -n 8 kind-config.yaml`	Shows the appended block.



Terminal 1 · captured 2026-10-23 12:27:33 UTC

> **Observation:** Without this line, kind create cluster failed here with "failed to update /proc/self/oom_score_adj: Permission denied". It does not change anything the lab teaches.

Figure 5. Building the 3-node cluster

Command	Purpose
`kind create cluster --config kind-config.yaml`	Creates three Docker containers that act as Kubernetes nodes, starts the control plane, installs networking (CNI) and storage, and joins the workers.

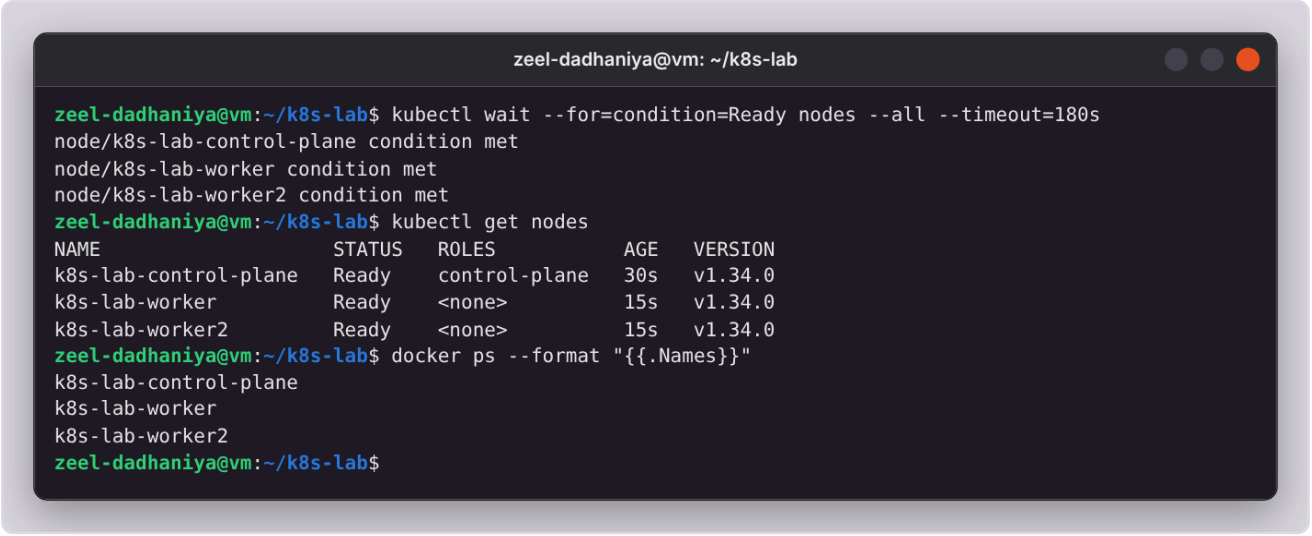


Terminal 1 · captured 2026-10-23 12:28:02 UTC

> **Observation:** Every stage completed and kubectl was switched to the new context kind-k8s-lab.

Figure 6. Three nodes in Kubernetes = three containers in Docker

Command	Purpose
`kubectl wait --for=condition=Ready nodes --all --timeout=180s`	Waits until every node reports Ready.
`kubectl get nodes`	Lists the cluster's nodes, their roles and their versions.
`docker ps --format "{{.Names}}"`	Lists only the names of the running Docker containers.

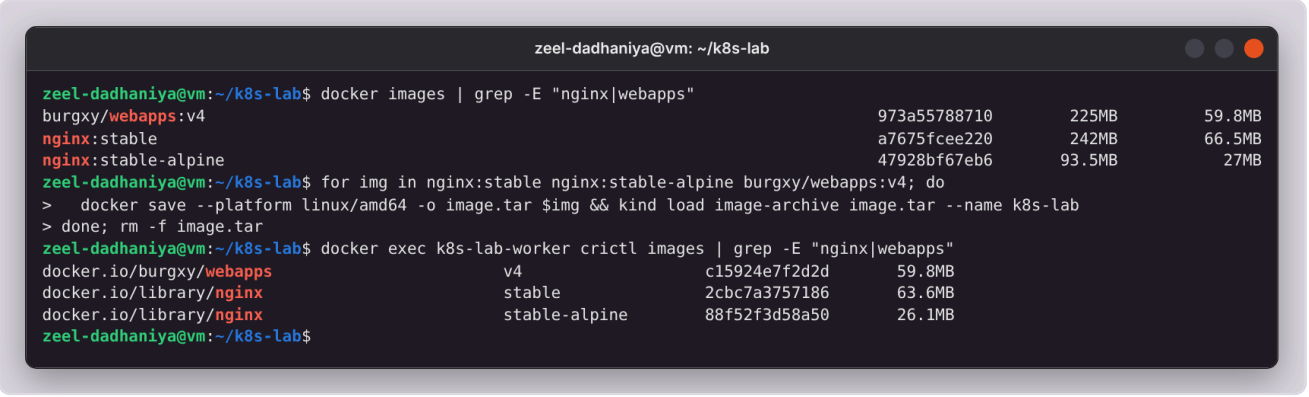


Terminal 1 · captured 2026-10-23 12:28:16 UTC

> **Observation:** Kubernetes reports k8s-lab-control-plane, k8s-lab-worker and k8s-lab-worker2 as Ready. Docker shows the same three names, because each node is a container.

Figure 7. Environment fix: loading the lab images into the nodes

Command	Purpose
`docker images | grep -E "nginx|webapps"`	Shows the three images the lab uses, already pulled by the host's Docker.
`docker save --platform linux/amd64 -o image.tar $img && kind load image-archive image.tar --name k8s-lab`	Exports each image to a tar file and imports it into every kind node, so the Pods can start without downloading.
`docker exec k8s-lab-worker crictl images | grep -E "nginx|webapps"`	Lists the images inside a worker node's container runtime to prove they arrived.



Terminal 1 · captured 2026-10-23 12:28:35 UTC

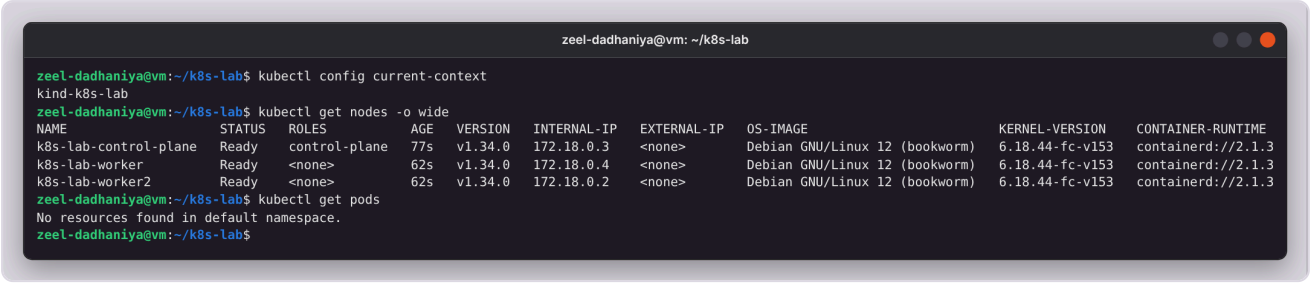
> **Observation:** nginx:stable, nginx:stable-alpine and burgxy/webapps:v4 are present on the nodes. The Pod definitions are unchanged; only the download step moved to the host.

Step 1 — Look at your cluster

Meet the three machines and see Kubernetes running itself.

Figure 8. Context, nodes and Pods

Command	Purpose
`kubectl config current-context`	Shows which cluster kubectl is talking to. This should always be checked before deleting anything.
`kubectl get nodes -o wide`	Lists the nodes with extra columns: internal IP, OS image, kernel and container runtime.
`kubectl get pods`	Lists Pods in the default namespace.

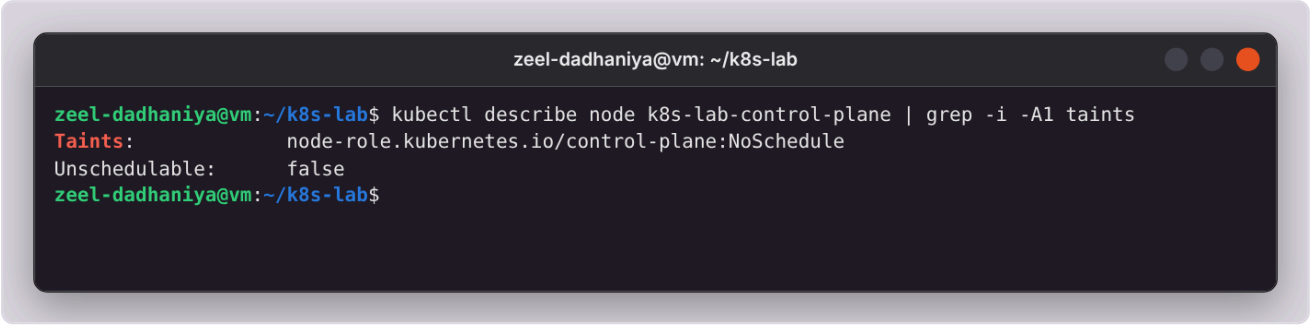


Terminal 1 · captured 2026-10-23 12:29:04 UTC

> **Observation:** The context is kind-k8s-lab. All 3 nodes are Ready with their own IPs (172.18.0.x). "No resources found" is correct because nothing has been deployed yet.

Figure 9. Why apps never run on the control plane

Command	Purpose
`kubectl describe node k8s-lab-control-plane | grep -i -A1 taints`	Shows the taints on the control-plane node, plus one line after.

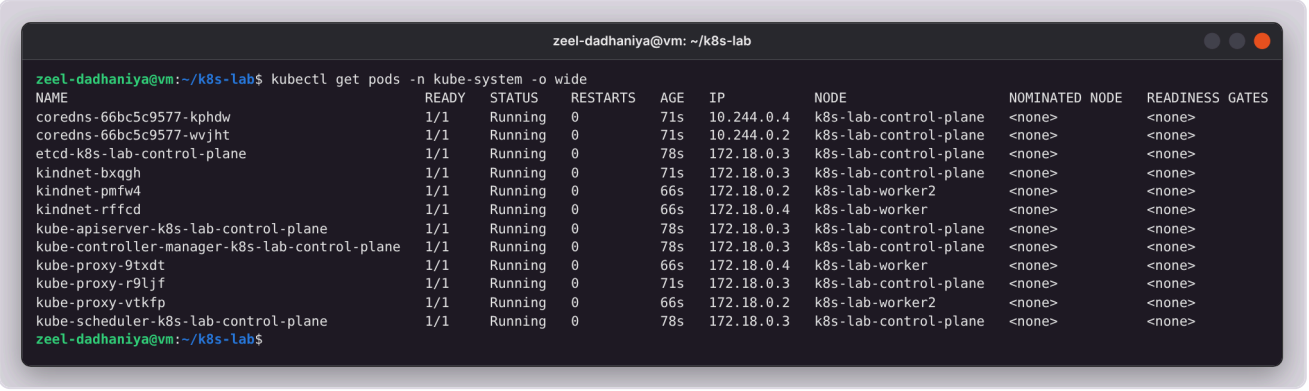


Terminal 1 · captured 2026-10-23 12:29:05 UTC

> **Observation:** Taint node-role.kubernetes.io/control-plane:NoSchedule is a "keep out" sign. Application Pods have no toleration for it, so they are only scheduled on the workers.

Figure 10. Kubernetes runs its own parts as Pods

Command	Purpose
`kubectl get pods -n kube-system -o wide`	Lists the system Pods in the kube-system namespace and shows which node each one runs on.



Terminal 1 · captured 2026-10-23 12:29:07 UTC

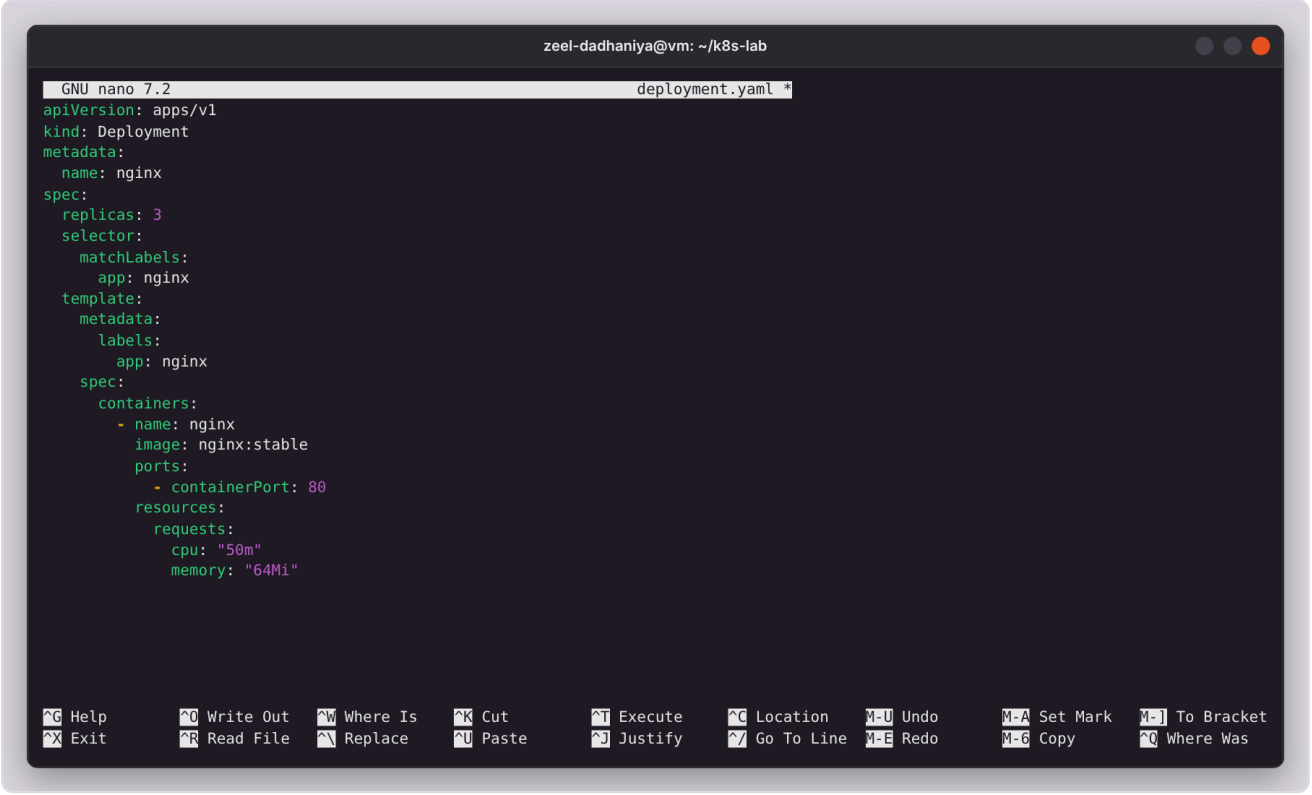
> **Observation:** kube-apiserver, etcd, kube-scheduler, kube-controller-manager and coredns run on the control plane. kube-proxy and kindnet run once on every node (3 copies each).

Step 2 — Run your first app

Write one Deployment file and let Kubernetes build and place everything.

Figure 11. Typing deployment.yaml in nano

Command	Purpose
`nano deployment.yaml`	Opens the nano editor. The Deployment was typed with spaces only (no tabs), 2 spaces per level.

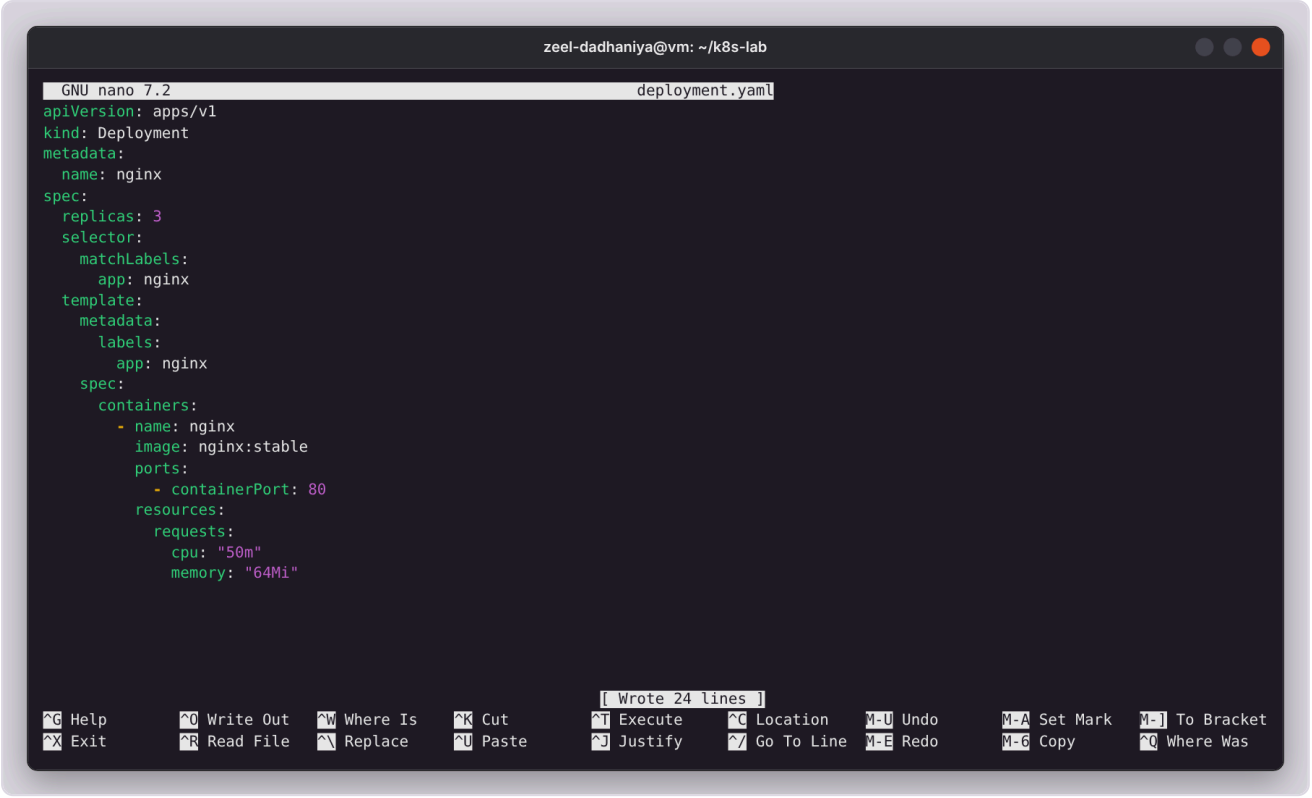


Terminal 1 · captured 2026-10-23 12:29:32 UTC

> **Observation:** kind: Deployment, replicas: 3, the matching app: nginx labels in selector and template, image nginx:stable on port 80, and requests of 50m CPU / 64Mi memory.

Figure 12. Saving the file

Command	Purpose
`Ctrl+O, Enter, then Ctrl+X`	Writes the file to disk, then exits nano.

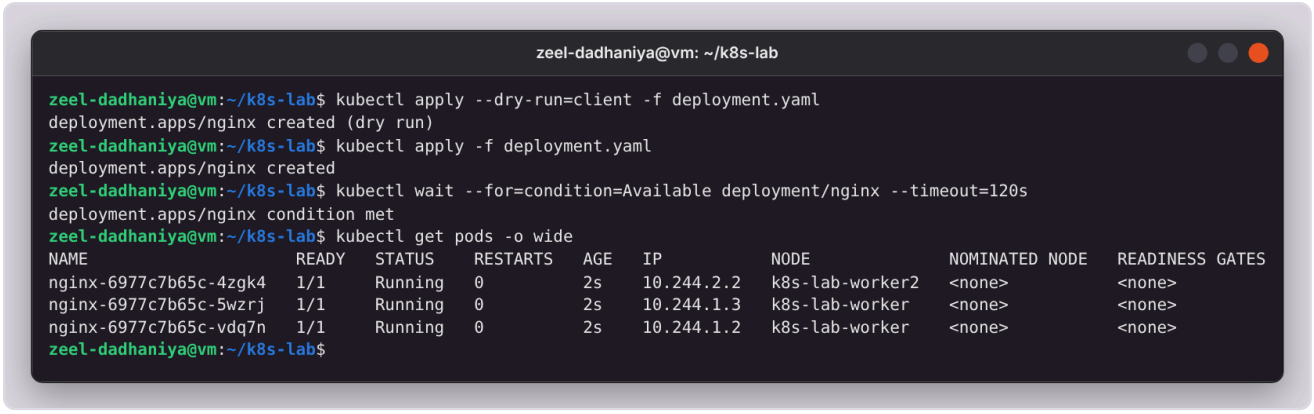


Terminal 1 · captured 2026-10-23 12:29:34 UTC

> **Observation:** nano confirms "Wrote 24 lines".

Figure 13. Validate, apply, wait, inspect

Command	Purpose
`kubectl apply --dry-run=client -f deployment.yaml`	Validates the file locally without creating anything.
`kubectl apply -f deployment.yaml`	Sends the desired state to the API server, which stores it in etcd.
`kubectl wait --for=condition=Available deployment/nginx --timeout=120s`	Waits until the Deployment is really available instead of guessing.
`kubectl get pods -o wide`	Shows the Pods with their IPs and the node each one runs on.

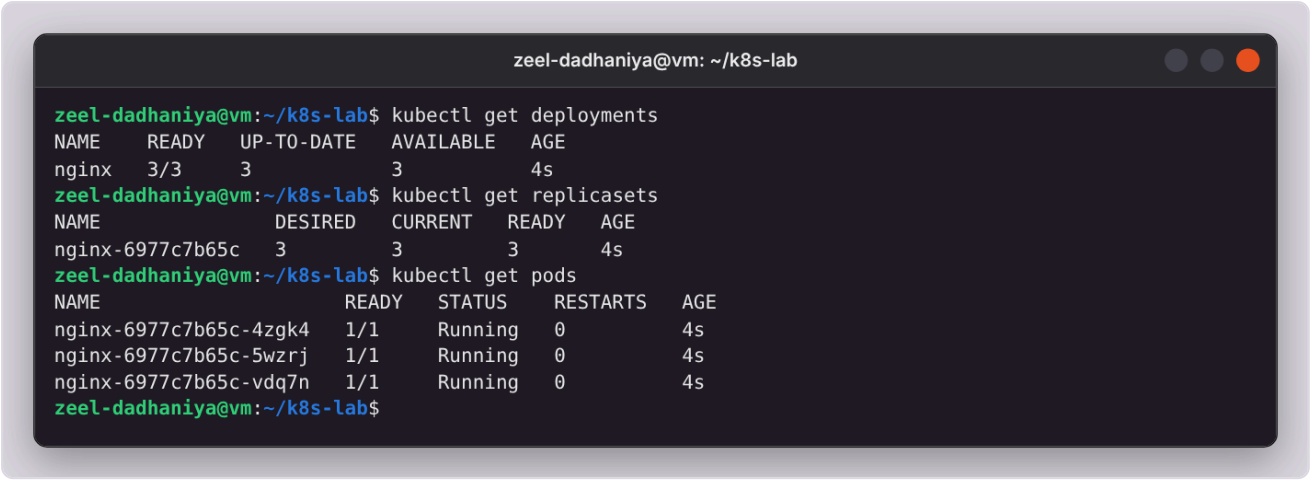


Terminal 1 · captured 2026-10-23 12:29:52 UTC

> **Observation:** 3 Pods are Running, spread over k8s-lab-worker and k8s-lab-worker2. No node was chosen by hand; the scheduler decided.

Figure 14. One file, three objects

Command	Purpose
`kubectl get deployments`	Shows the Deployment: the object that was written.
`kubectl get replicasets`	Shows the ReplicaSet the Deployment created. It keeps exactly 3 Pods.
`kubectl get pods`	Shows the Pods the ReplicaSet created.

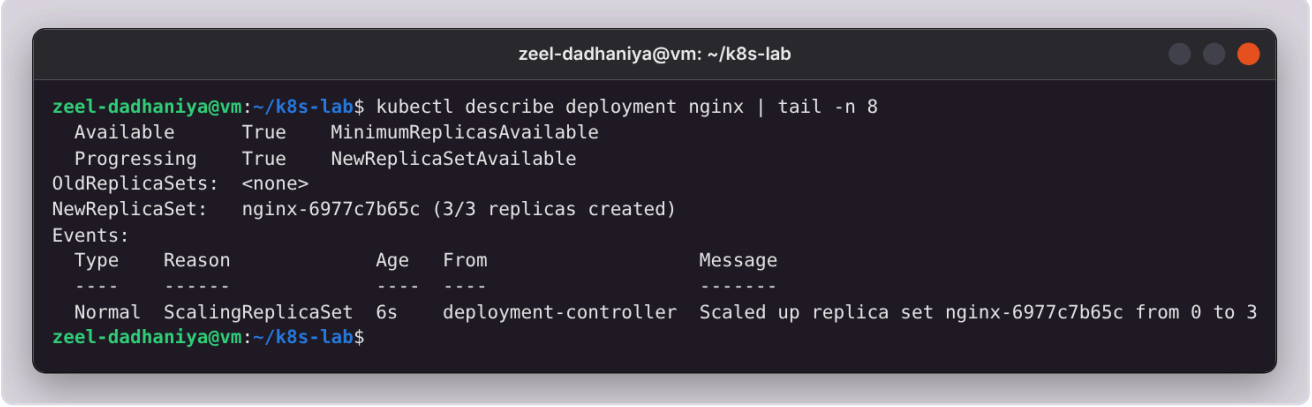


Terminal 1 · captured 2026-10-23 12:29:55 UTC

> **Observation:** Names show the chain: Deployment nginx → ReplicaSet nginx-6977c7b65c → Pods nginx-6977c7b65c-xxxxx. The code 6977c7b65c is a hash of the Pod template.

Figure 15. The Deployment's Events

Command	Purpose
`kubectl describe deployment nginx | tail -n 8`	Shows the last lines of the description: conditions, the active ReplicaSet and the Events.



Terminal 1 · captured 2026-10-23 12:29:57 UTC

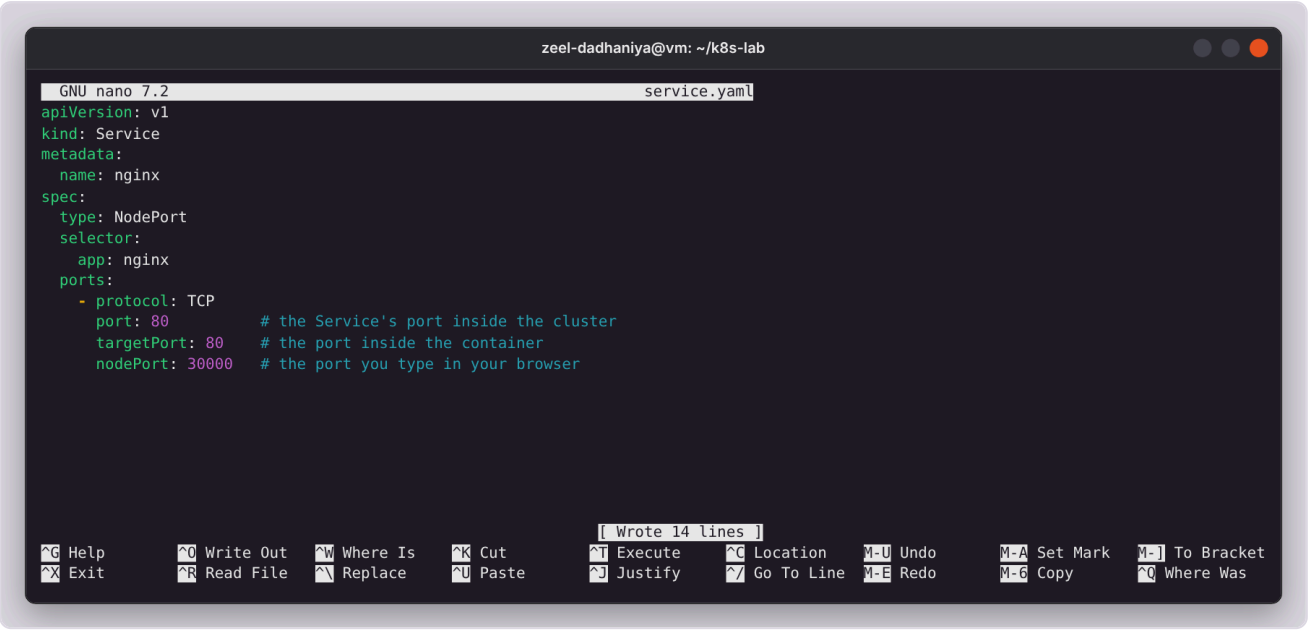
> **Observation:** Event ScalingReplicaSet by deployment-controller: "Scaled up replica set nginx-6977c7b65c from 0 to 3". This is the hand-over from the Deployment to its ReplicaSet.

Step 3 — Open it in a browser

Put one fixed address (a Service) in front of the moving Pods and reach it on a real port.

Figure 16. Writing service.yaml in nano

Command	Purpose
`nano service.yaml`	Creates the Service file (apiVersion v1, type NodePort, selector app: nginx, port 80 → targetPort 80, nodePort 30000).

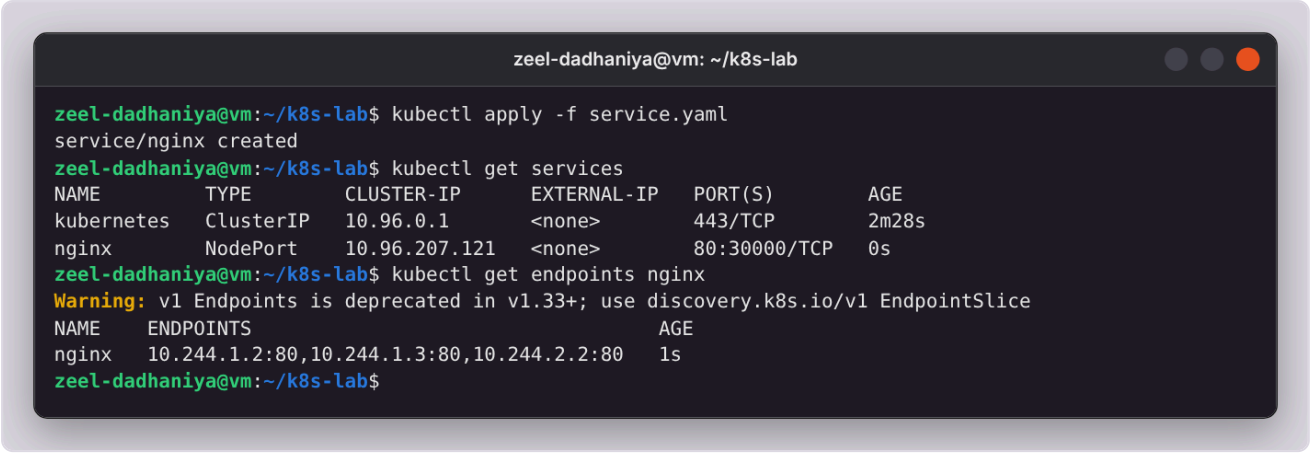


Terminal 1 · captured 2026-10-23 12:30:12 UTC

> **Observation:** The selector app: nginx matches the Pod label in deployment.yaml exactly.

Figure 17. Creating the Service and checking its endpoints

Command	Purpose
`kubectl apply -f service.yaml`	Creates the Service.
`kubectl get services`	Lists Services with their cluster IP and ports.
`kubectl get endpoints nginx`	Shows which Pod IPs the Service sends traffic to.

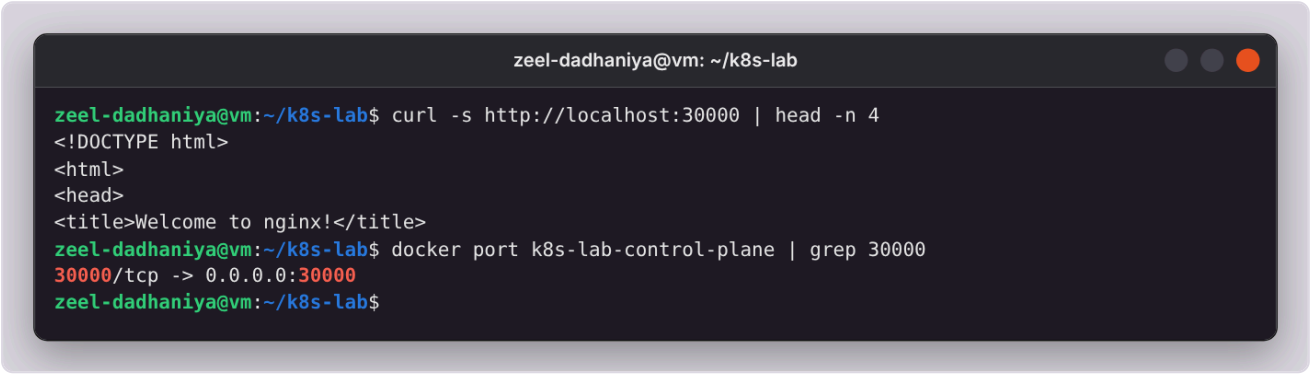


Terminal 1 · captured 2026-10-23 12:30:16 UTC

Observation: nginx is a NodePort Service (80:30000/TCP). ENDPOINTS lists exactly the 3 Pod IPs, which proves the labels match. (Kubernetes 1.33+ prints a deprecation warning for v1 Endpoints; the command still works.)

Figure 18. Testing the NodePort from the terminal

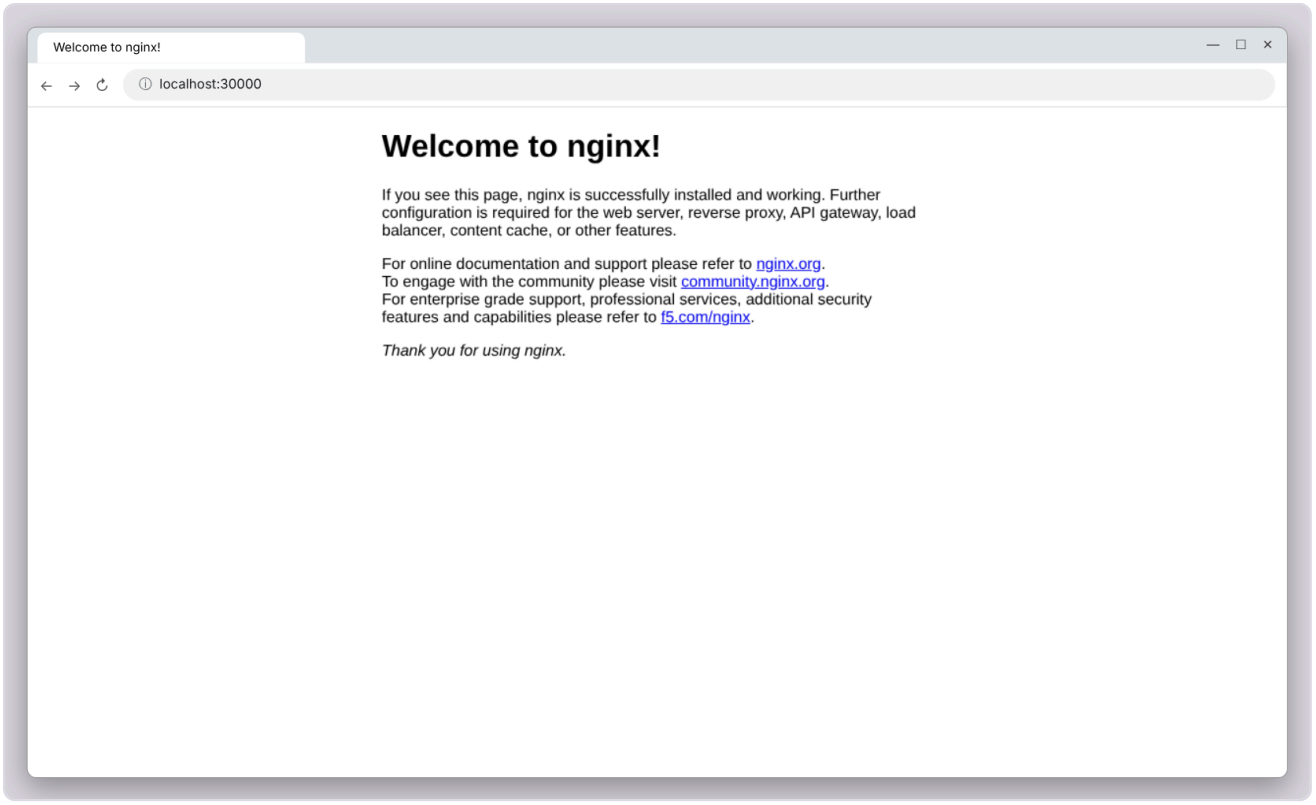
Command	Purpose
`curl -s http://localhost:30000 | head -n 4`	Requests the page through the NodePort and prints the first lines of the HTML.
`docker port k8s-lab-control-plane | grep 30000`	Confirms that host port 30000 is mapped into the control-plane container.



Terminal 1 · captured 2026-10-23 12:30:18 UTC

Observation: The response title is "Welcome to nginx!".

Figure 19. Browser: http://localhost:30000

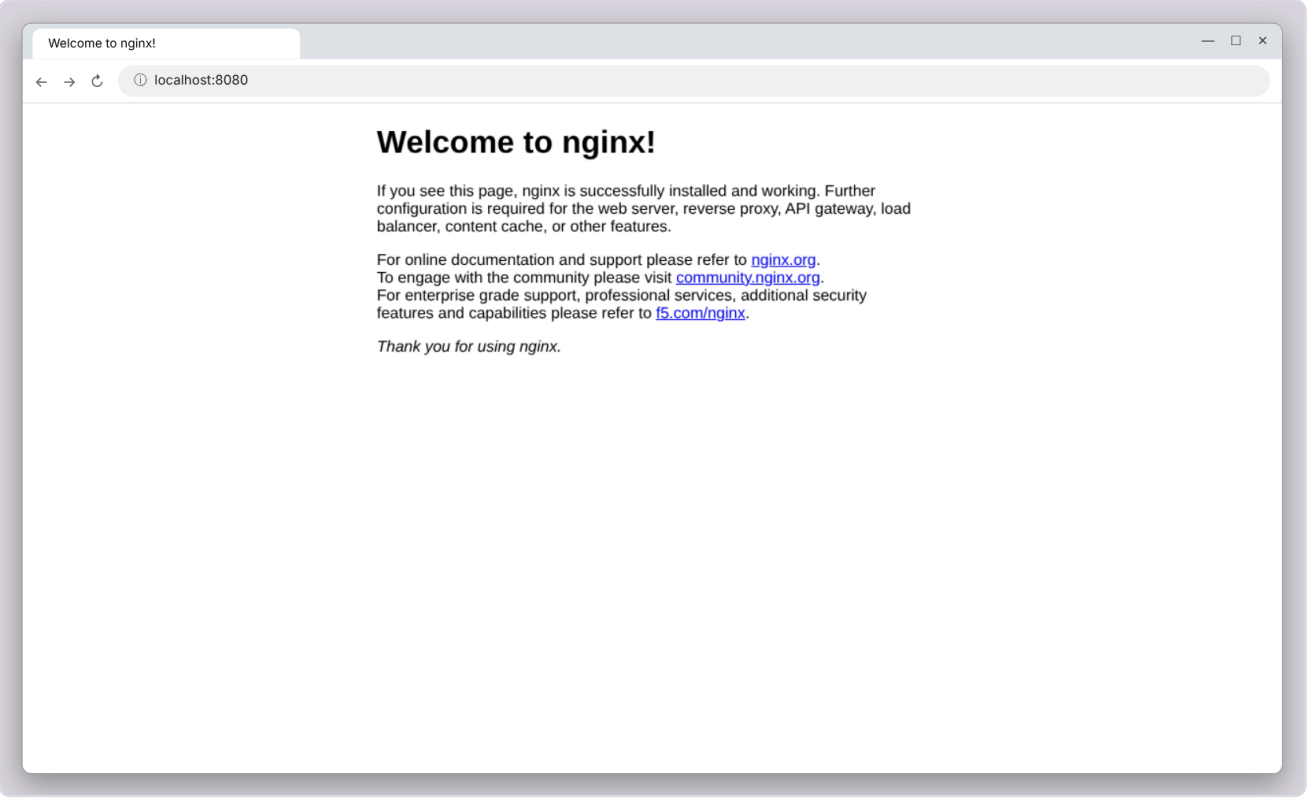


Browser · captured 2026-10-23 12:30:26 UTC

Observation: The "Welcome to nginx!" page loads through the NodePort with no tunnel held open.

Figure 20. Browser: http://localhost:8080 through port-forward

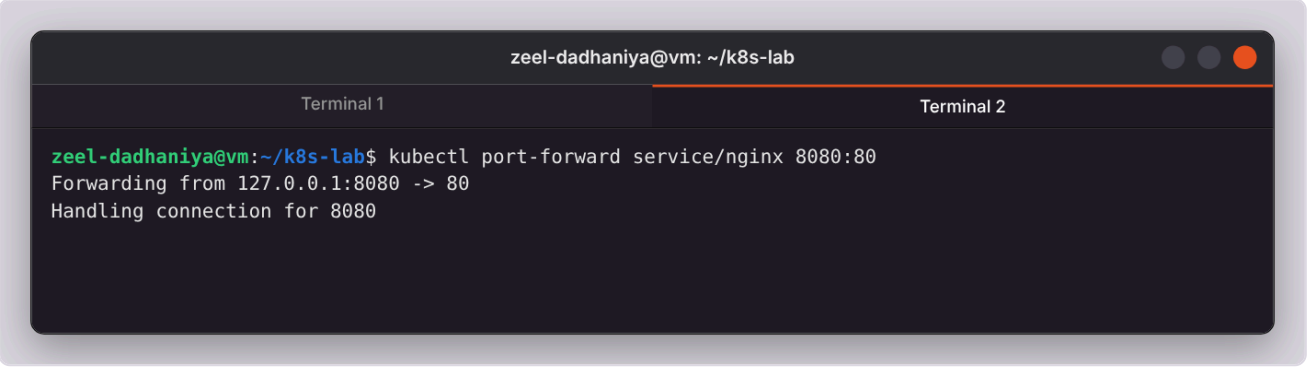
Command	Purpose
`kubectl port-forward service/nginx 8080:80 (Terminal 2)`	Opens a temporary tunnel from localhost:8080 to the Service's port 80. It runs until stopped.



Browser · captured 2026-10-23 12:30:32 UTC

> **Observation:** The same nginx page is served on port 8080 while the tunnel is open.

Figure 21. Terminal 2: the tunnel in action

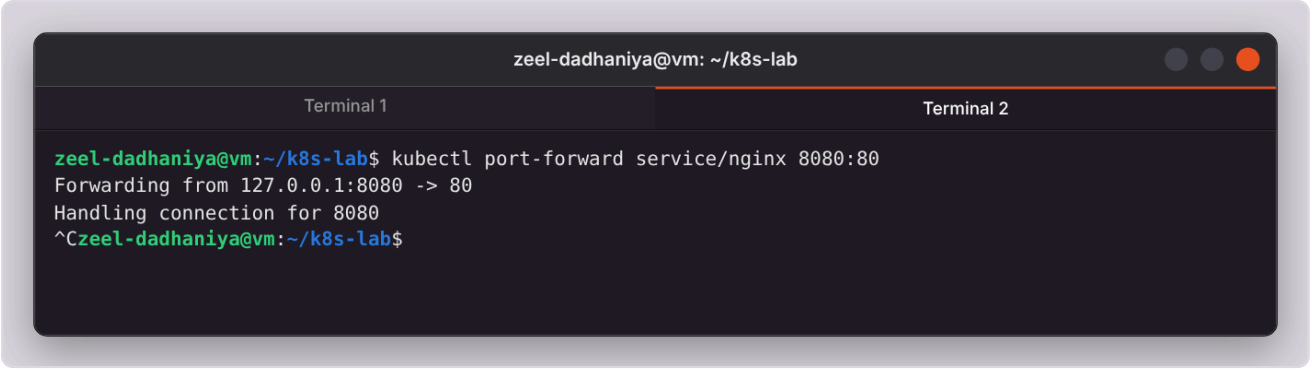


Terminal 2 · captured 2026-10-23 12:30:33 UTC

> **Observation:** "Forwarding from 127.0.0.1:8080 → 80". "Handling connection for 8080" appeared when the browser opened the page.

Figure 22. Terminal 2: stopping the tunnel

Command	Purpose
`Ctrl+C`	Stops kubectl port-forward and closes the tunnel.



Terminal 2 · captured 2026-10-23 12:30:35 UTC

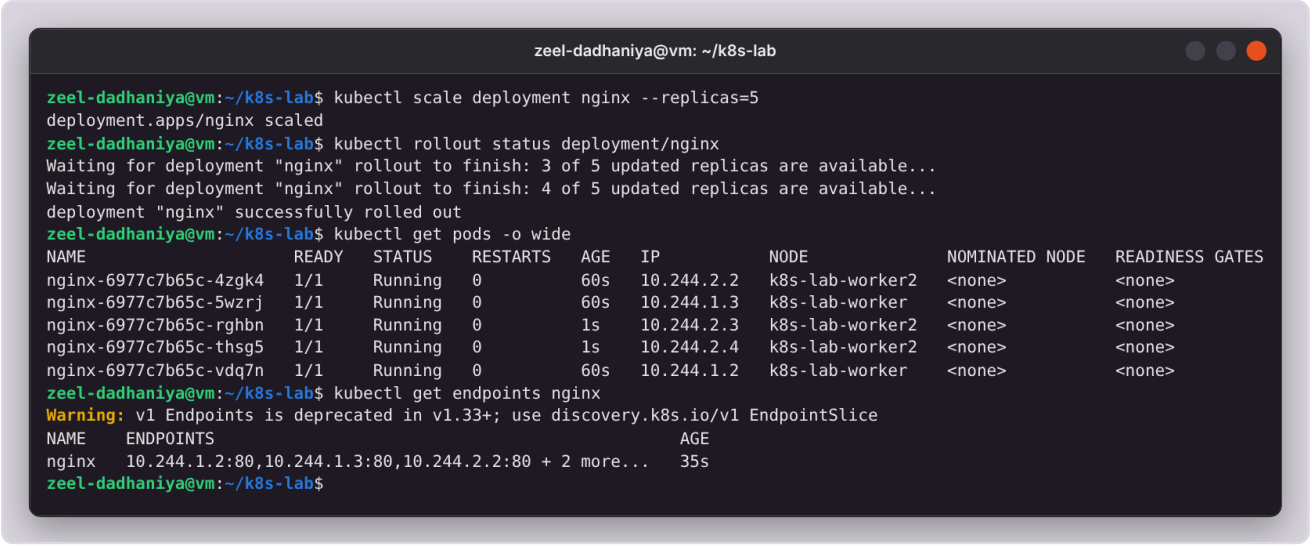
> **Observation:** port-forward is for a quick private look. The NodePort keeps working after the tunnel is closed.

Step 4 — Scale it, then kill it

Turn the replica dial up and down, then delete a running Pod and watch Kubernetes replace it.

Figure 23. Scaling up to 5 replicas

Command	Purpose
`kubectl scale deployment nginx --replicas=5`	Changes the desired number of Pods from 3 to 5.
`kubectl rollout status deployment/nginx`	Waits until all 5 replicas are available.
`kubectl get pods -o wide`	Shows the old and new Pods.
`kubectl get endpoints nginx`	Shows that the Service picked up the new Pods.

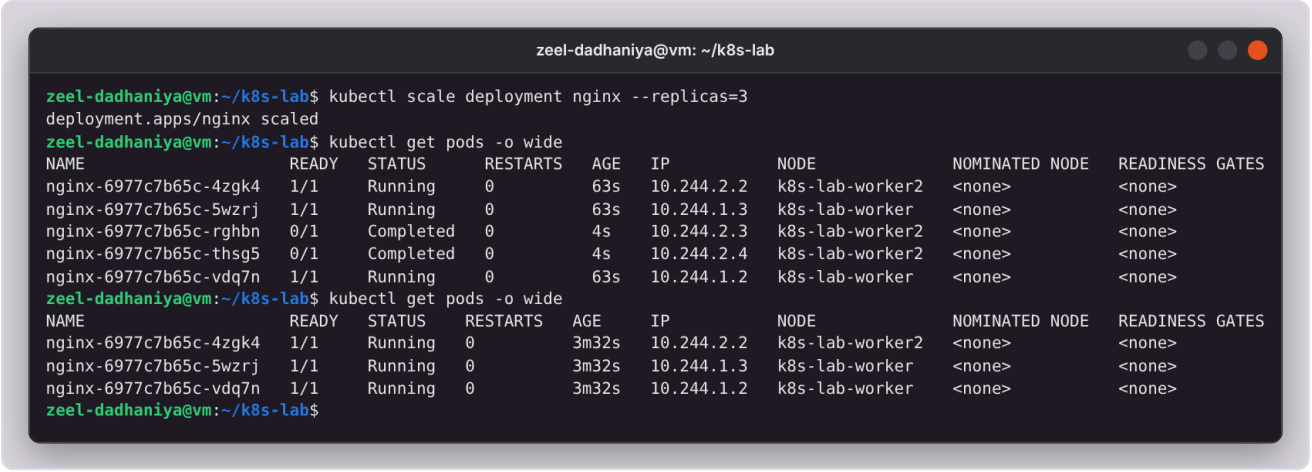


Terminal 1 · captured 2026-10-23 12:30:51 UTC

> **Observation:** Two new Pods (AGE 1s) were added; the original three (AGE 60s) were not restarted. The endpoints grew to 5 ("+ 2 more...") without touching service.yaml.

Figure 24. Scaling back to 3

Command	Purpose
`kubectl scale deployment nginx --replicas=3`	Sets the desired count back to 3.
`kubectl get pods -o wide`	Run twice: right away (2 Pods shutting down) and a few seconds later.

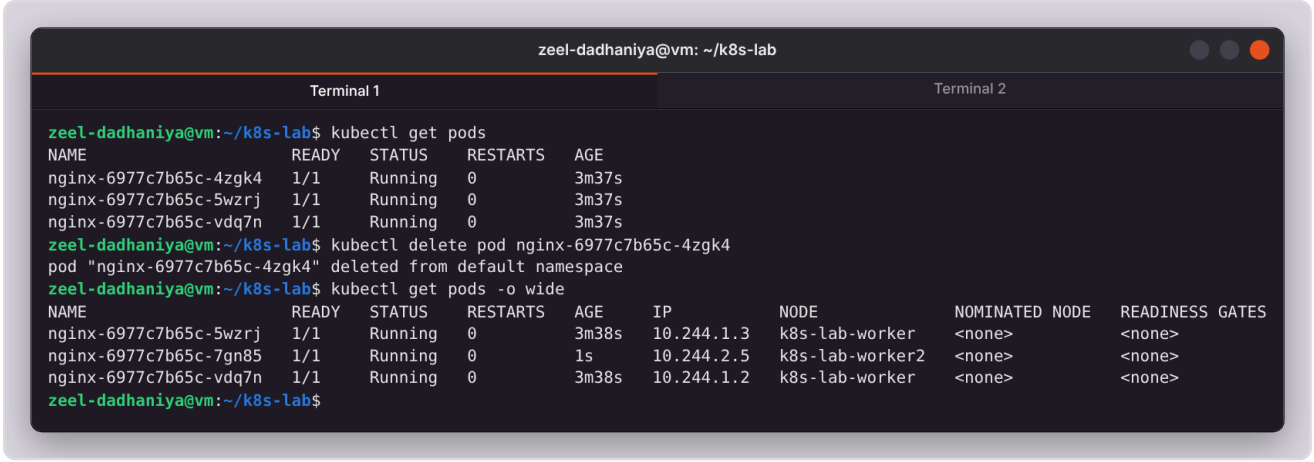


Terminal 1 · captured 2026-10-23 12:33:23 UTC

> **Observation:** The two extra Pods were stopped and exactly 3 remain.

Figure 25. Terminal 1: deleting a running Pod on purpose

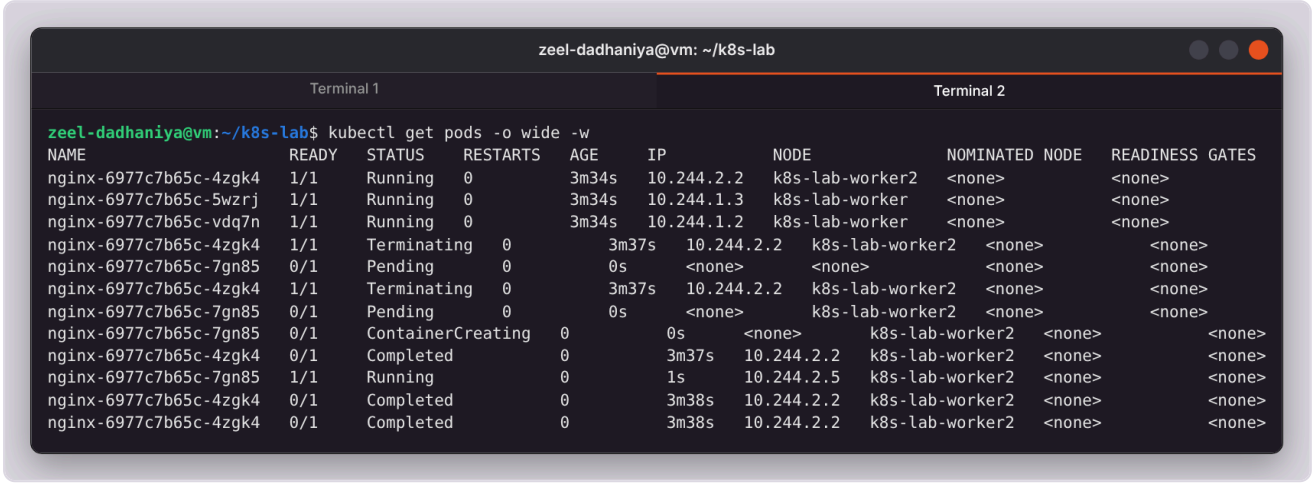
Command	Purpose
`kubectl get pods -o wide -w (Terminal 2)`	Started first in the second tab to watch changes live.
`kubectl get pods`	Lists the Pod names so one can be copied.
`kubectl delete pod nginx-6977c7b65c-4zgk4`	Deletes one running Pod.
`kubectl get pods -o wide`	Checks the result right away.



Terminal 1 · captured 2026-10-23 12:33:29 UTC

> **Observation:** A replacement Pod nginx-6977c7b65c-7gn85 was already Running (AGE 1s) with a new name and a new IP (10.244.2.5).

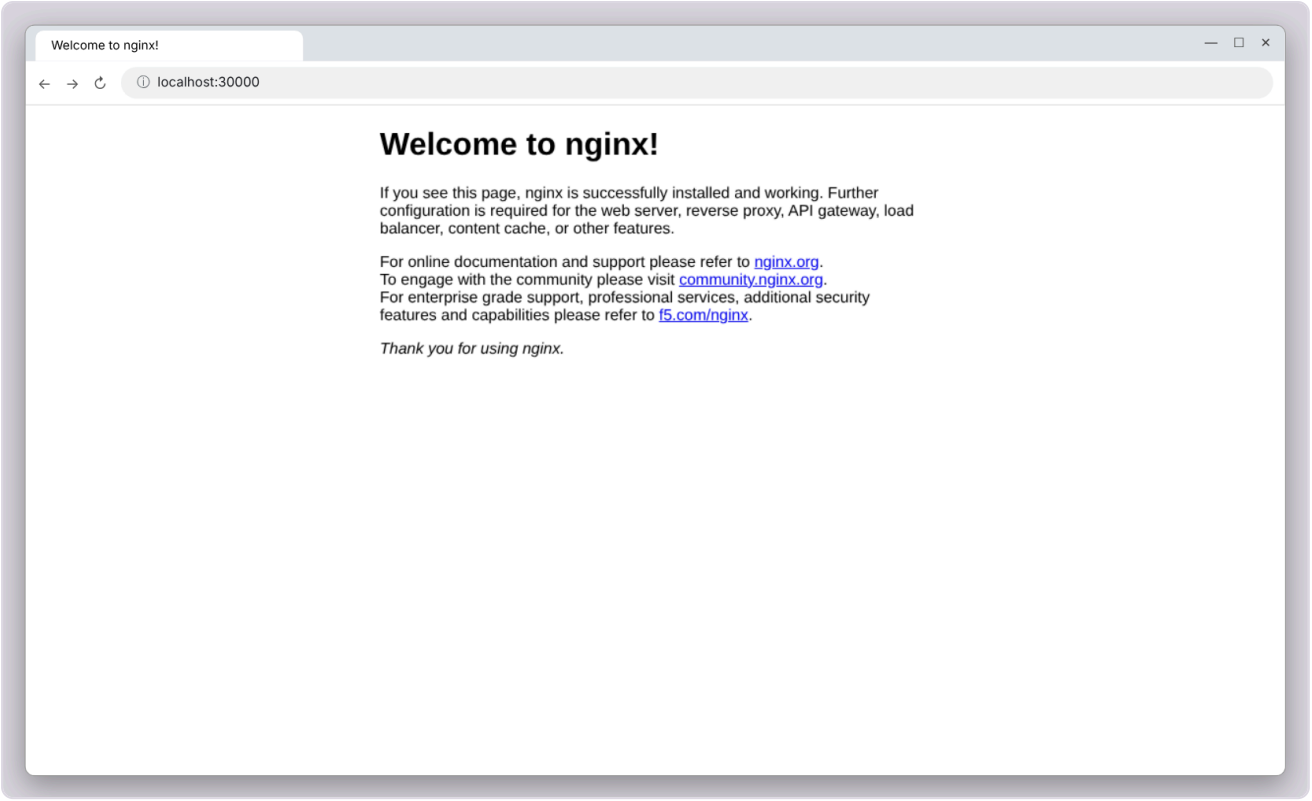
Figure 26. Terminal 2: the live watch



Terminal 2 · captured 2026-10-23 12:33:32 UTC

> **Observation:** The watch shows 4zgk4 Terminating, a new Pod 7gn85 going Pending → ContainerCreating → Running within about 1–2 seconds. The ReplicaSet counted 2, wanted 3, and made one more.

Figure 27. Browser after the deletion



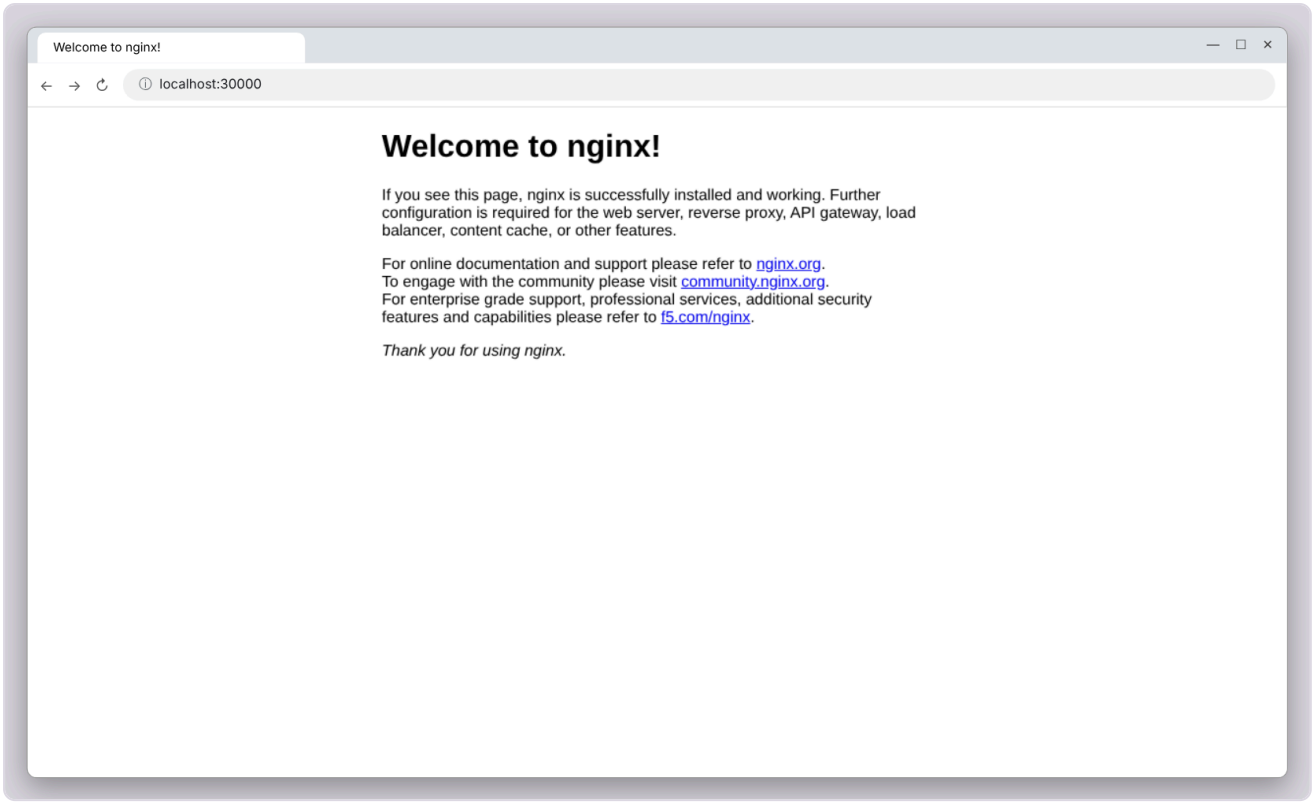
Browser · captured 2026-10-23 12:33:41 UTC

> **Observation:** The site never went down: the other Pods served requests while the replacement started.

Step 5 — Update, redeploy, undo

Ship new versions with rolling updates, redeploy without changing the file, and roll back.

Figure 28. Browser during and after the imperative update

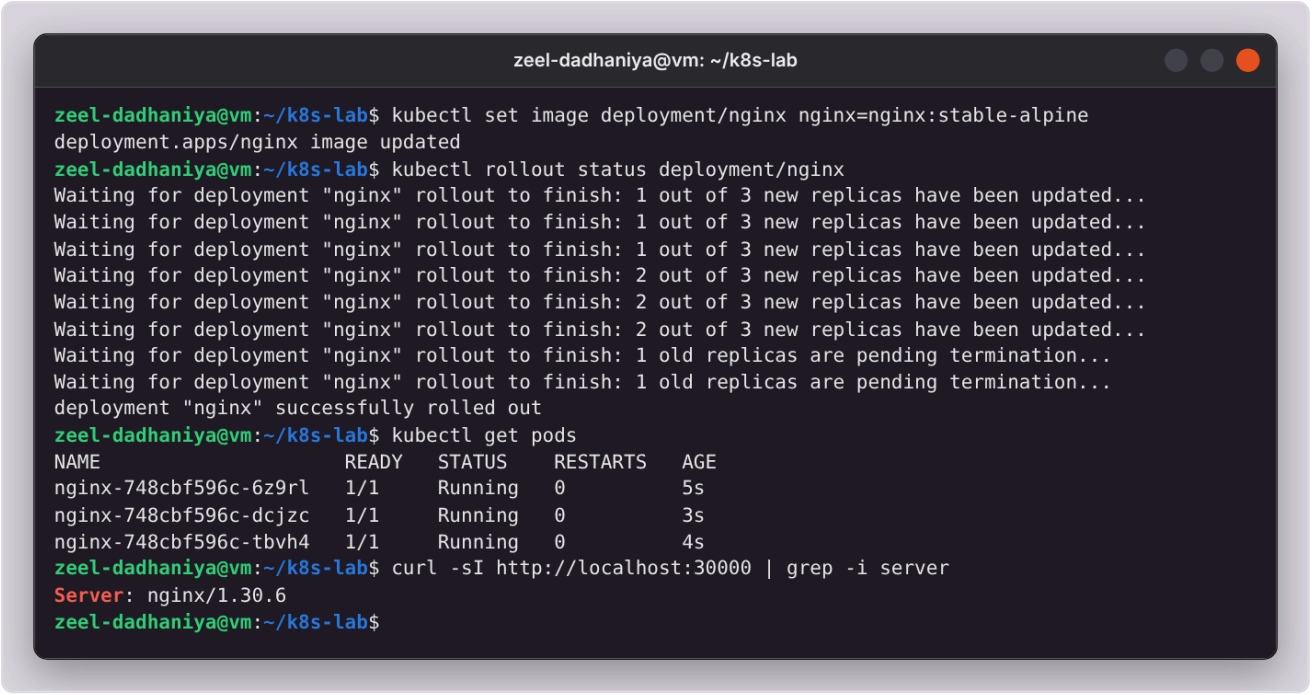


Browser · captured 2026-10-23 12:33:46 UTC

> **Observation:** The page stays available while the Pods are replaced one by one.

Figure 29. 5.1 Imperative update: kubectl set image

Command	Purpose
`kubectl set image deployment/nginx nginx=nginx:stable-alpine`	Changes the live Deployment's container named nginx to the image nginx:stable-alpine.
`kubectl rollout status deployment/nginx`	Follows the rolling update until it finishes.
`kubectl get pods`	Shows the new Pods (new ReplicaSet hash 748cbf596c).
`curl -sI http://localhost:30000 | grep -i server`	Reads the Server header from the running version.

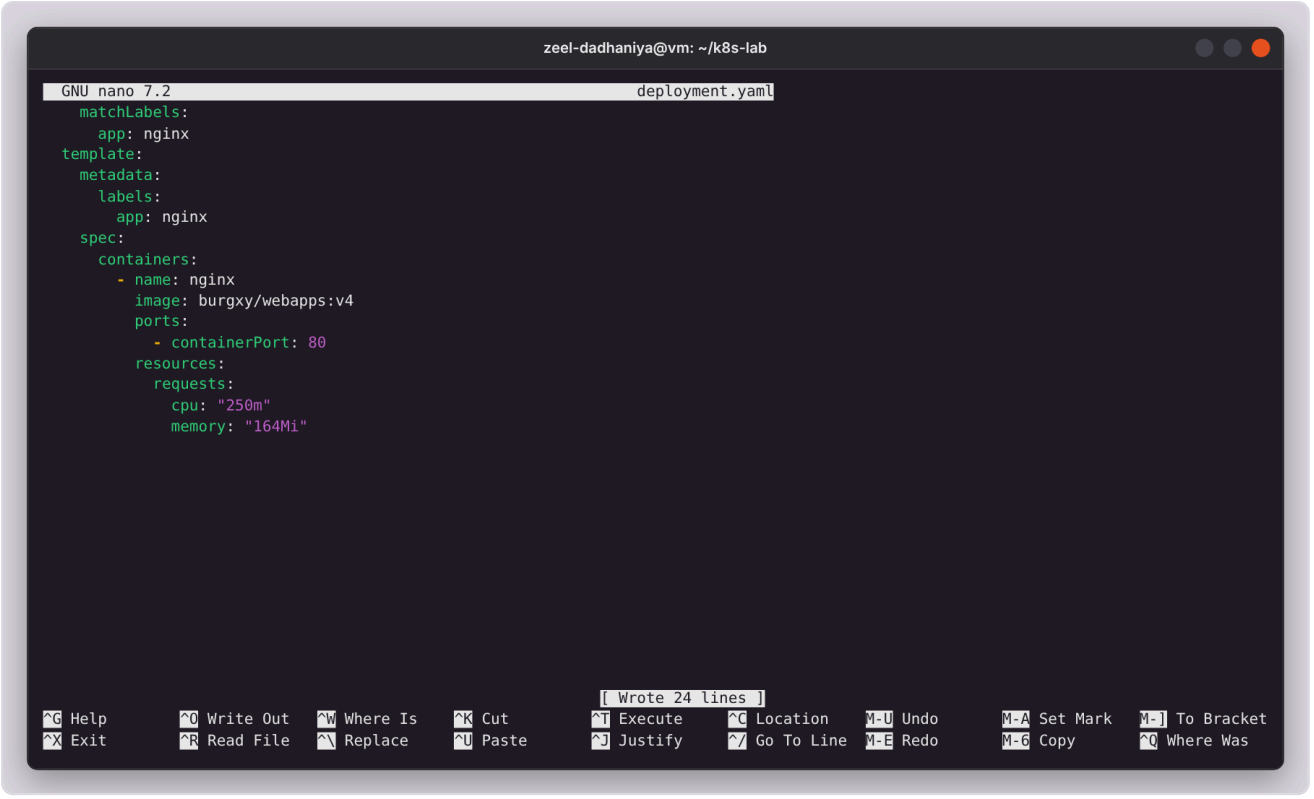


Terminal 1 · captured 2026-10-23 12:33:49 UTC

> **Observation:** The rollout replaced the Pods one at a time (1 of 3, 2 of 3, then the old ones terminated) and finished successfully.

Figure 30. 5.1 Declarative update: editing deployment.yaml

Command	Purpose
`nano deployment.yaml (Ctrl+_ go to line, Ctrl+K cut line, type the new line, Ctrl+O save)`	Changes the image to burgxy/webapps:v4 and the requests to 250m CPU / 164Mi memory, as in the lab.

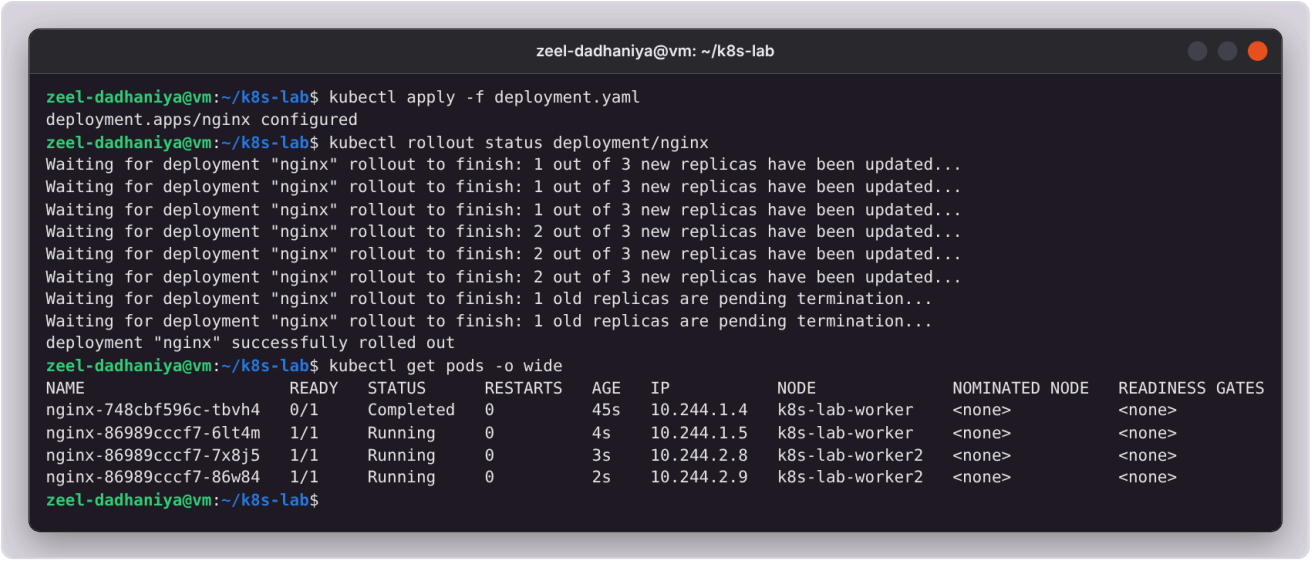


Terminal 1 · captured 2026-10-23 12:34:15 UTC

> **Observation:** The file now holds the new desired state. This is the production-style way: the change lives in a file that can be stored in Git.

Figure 31. 5.1 Applying the edited file

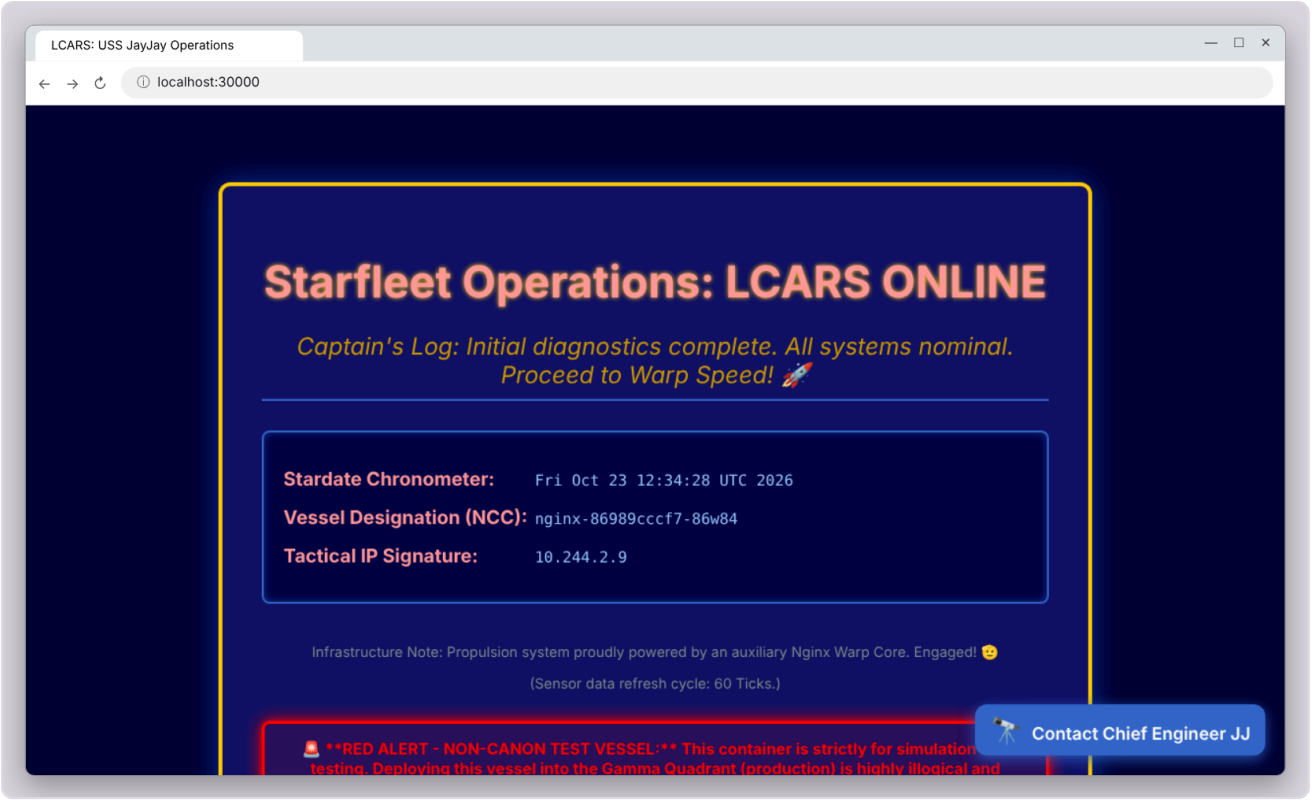
Command	Purpose
`kubectl apply -f deployment.yaml`	Compares the file with the cluster and applies the differences ("configured").
`kubectl rollout status deployment/nginx`	Waits for the rolling update to finish.
`kubectl get pods -o wide`	Shows Pods from the new ReplicaSet 86989cccf7.



Terminal 1 · captured 2026-10-23 12:34:29 UTC

> **Observation:** burgxy/webapps:v4 rolled out with no downtime. One old Pod is still shown finishing shutdown (Completed).

Figure 32. Browser: the new version

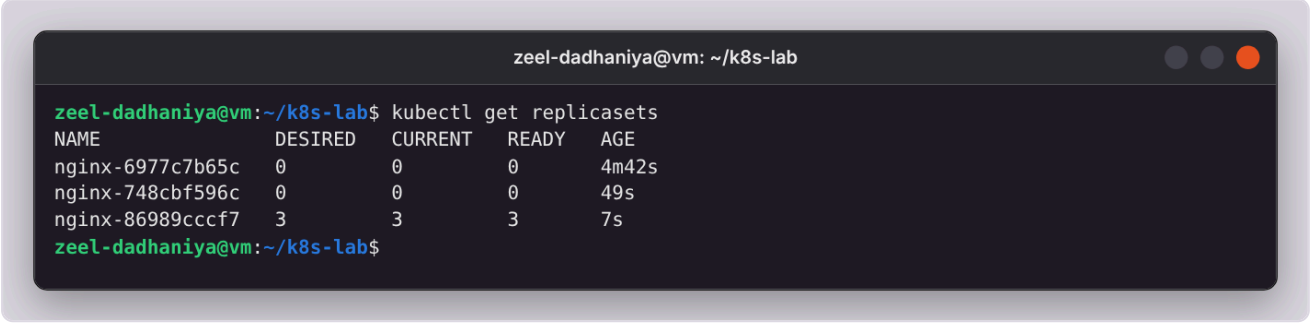


Browser · captured 2026-10-23 12:34:30 UTC

> **Observation:** The new app ("Starfleet Operations: LCARS ONLINE") is served. It prints the Pod name (nginx-86989cccf7-86w84) and the Pod IP that answered.

Figure 33. One ReplicaSet per version

Command	Purpose
`kubectl get replicasets`	Lists all ReplicaSets the Deployment has created.

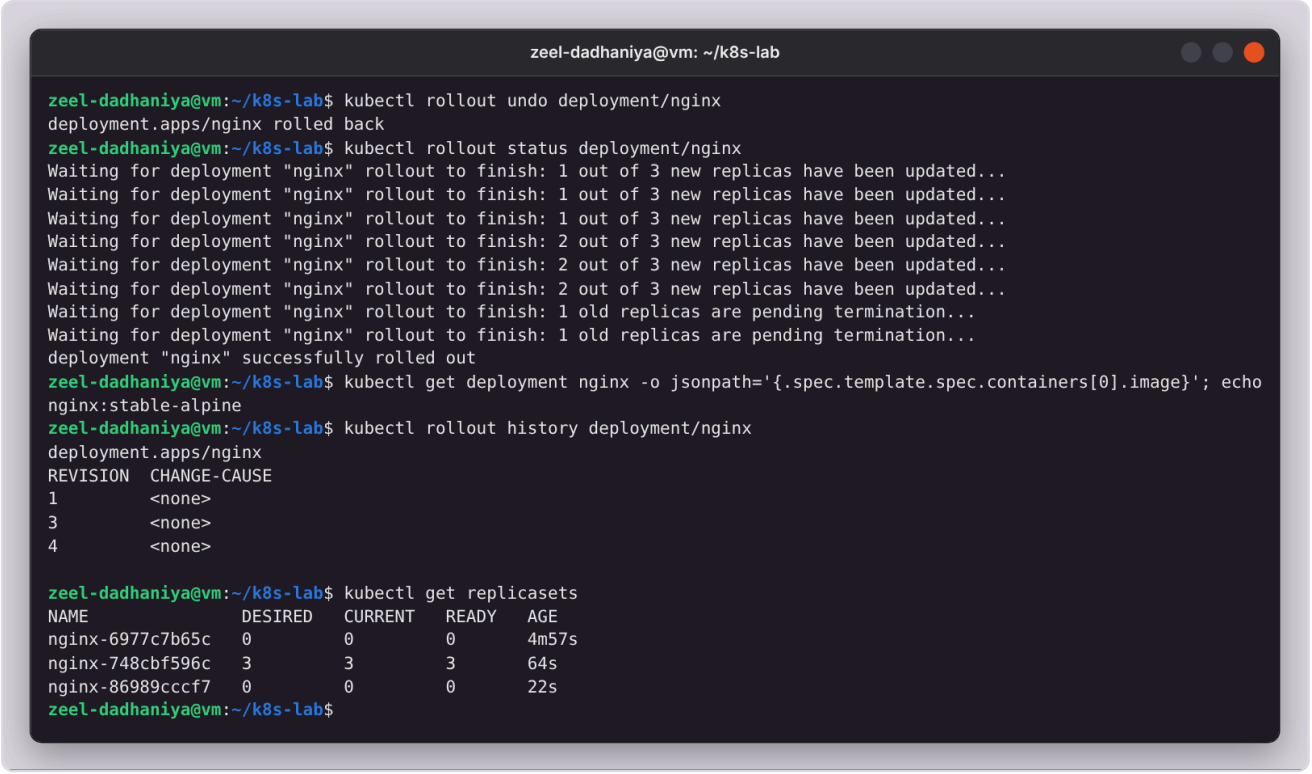


Terminal 1 · captured 2026-10-23 12:34:32 UTC

> **Observation:** 6977c7b65c (nginx:stable) and 748cbf596c (stable-alpine) are kept at 0 replicas. 86989cccf7 (webapps:v4) runs 3. The empty ones are the undo history.

Figure 34. Rolling back with rollout undo

Command	Purpose
`kubectl rollout undo deployment/nginx`	Rolls back to the previous revision.
`kubectl rollout status deployment/nginx`	Waits for the rollback to finish.
`kubectl get deployment nginx -o jsonpath='{.spec.template.spec.containers[0].image}'; echo`	Prints only the image currently in the Deployment's template.
`kubectl rollout history deployment/nginx`	Lists the recorded revisions (extra check).
`kubectl get replicasets`	Shows which ReplicaSet is active again.

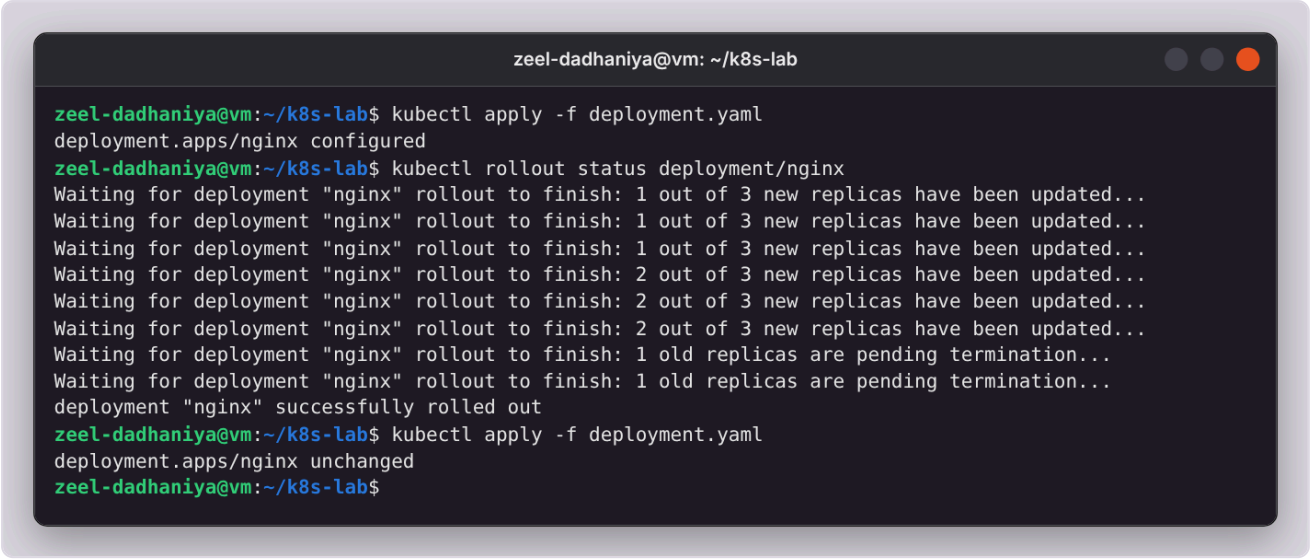


Terminal 1 · captured 2026-10-23 12:34:47 UTC

> **Observation:** The image went back to nginx:stable-alpine: undo returns to the revision just before the current one. The lab text says nginx:stable, but that assumes only one update; this lab does two (stable-alpine, then webapps:v4). The old ReplicaSet 748cbf596c was scaled up again, so nothing was rebuilt.

Figure 35. 5.2 Applying a file that did not change

Command	Purpose
`kubectl apply -f deployment.yaml (1st time)`	The live Deployment had been rolled back, so it no longer matched the file (webapps:v4). apply re-synced it: "configured".
`kubectl rollout status deployment/nginx`	Waits for that re-sync rollout.
`kubectl apply -f deployment.yaml (2nd time)`	Now the file and the cluster are the same, so apply reports "unchanged" and does nothing.

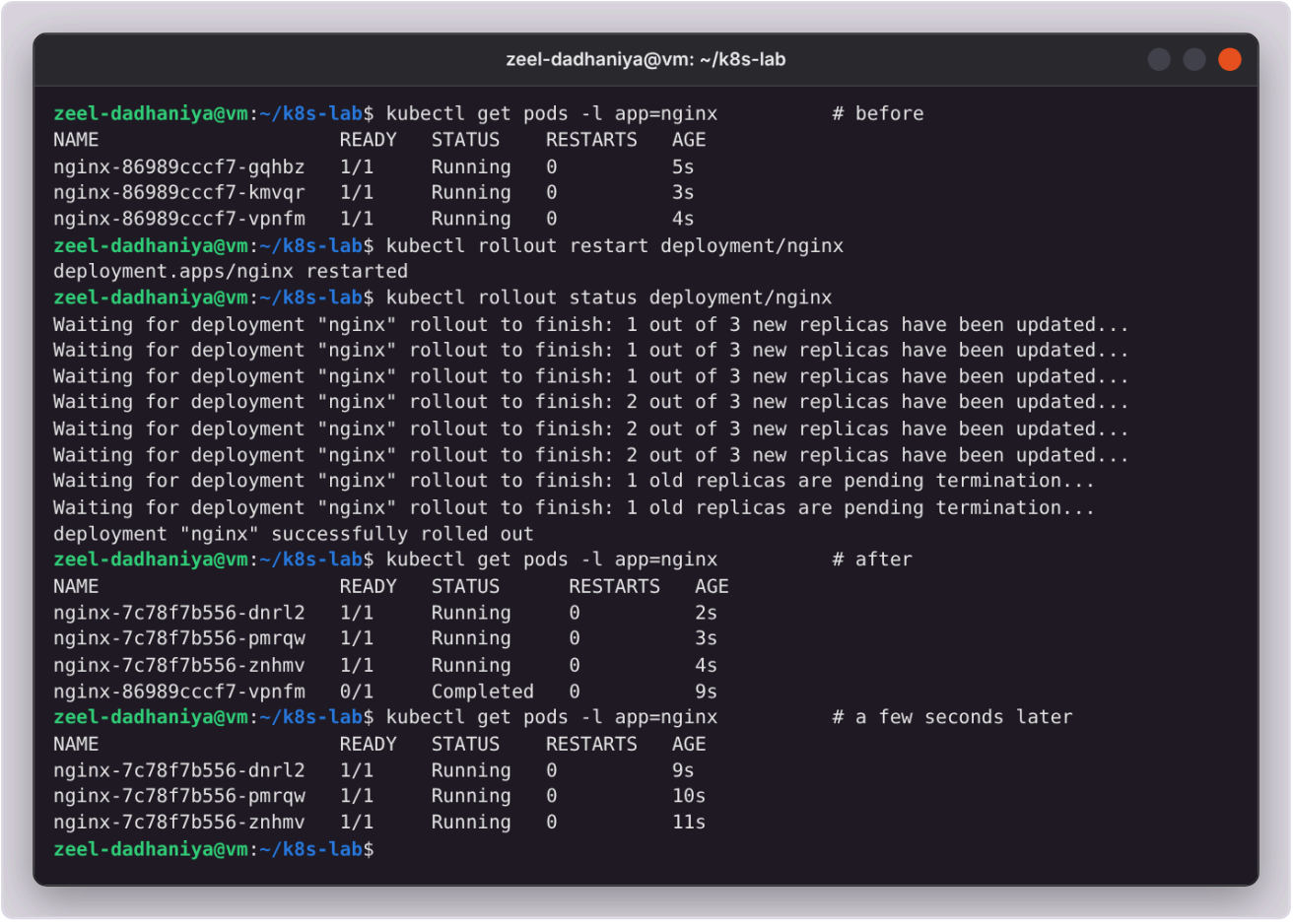


Terminal 1 · captured 2026-10-23 12:35:01 UTC

> **Observation:** This shows how declarative apply works: it acts only on differences. The first apply also shows that the file is the source of truth: it undid the manual rollback.

Figure 36. 5.2 Redeploying with rollout restart

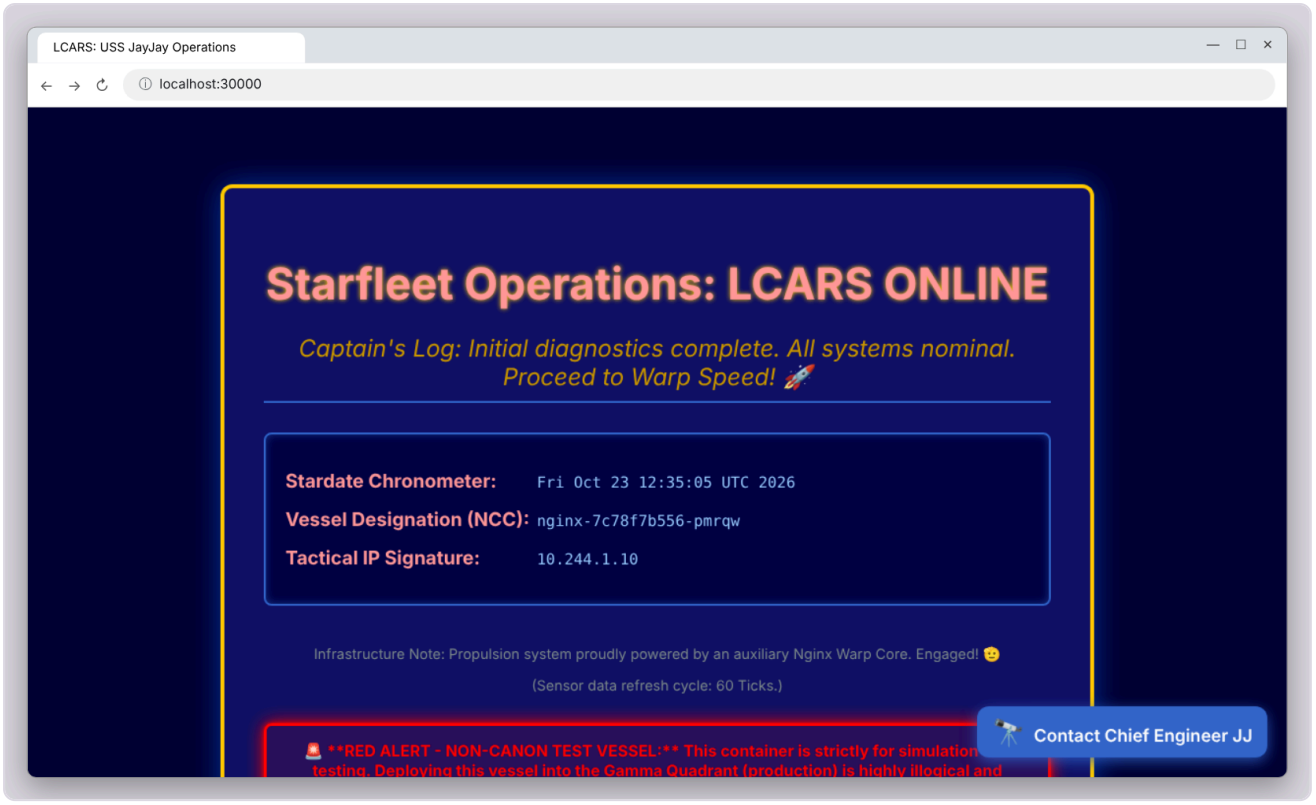
Command	Purpose
`kubectl get pods -l app=nginx # before`	Records the Pod names before the restart (hash 86989cccf7).
`kubectl rollout restart deployment/nginx`	Asks for every Pod to be replaced, even though nothing in the file changed.
`kubectl rollout status deployment/nginx`	Follows the rolling replacement.
`kubectl get pods -l app=nginx # after`	Run twice: right after, and a few seconds later once the last old Pod had gone.



Terminal 1 · captured 2026-10-23 12:35:15 UTC

> **Observation:** All Pods are new (hash 7c78f7b556, ages a second apart) and RESTARTS is still 0. The Pods were replaced, not restarted in place. The image and the YAML are unchanged.

Figure 37. Browser after the restart



Browser · captured 2026-10-23 12:35:15 UTC

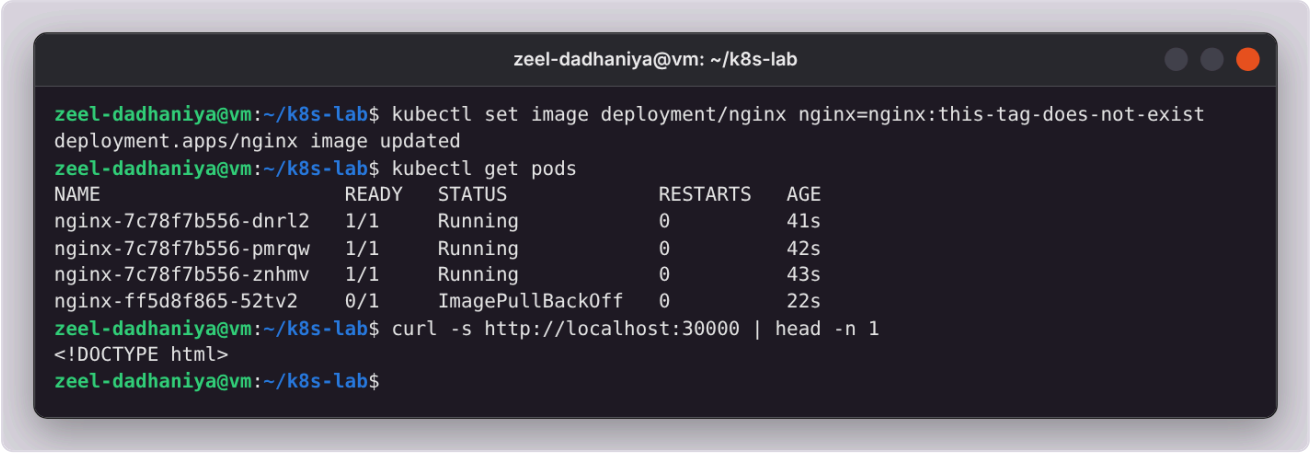
> **Observation:** The page is served by one of the new Pods; the site stayed up during the rolling replacement.

Step 6 — Break it and fix it

Cause a real failure, find the cause, and repair it.

Figure 38. 6.1 Breaking it with an image tag that does not exist

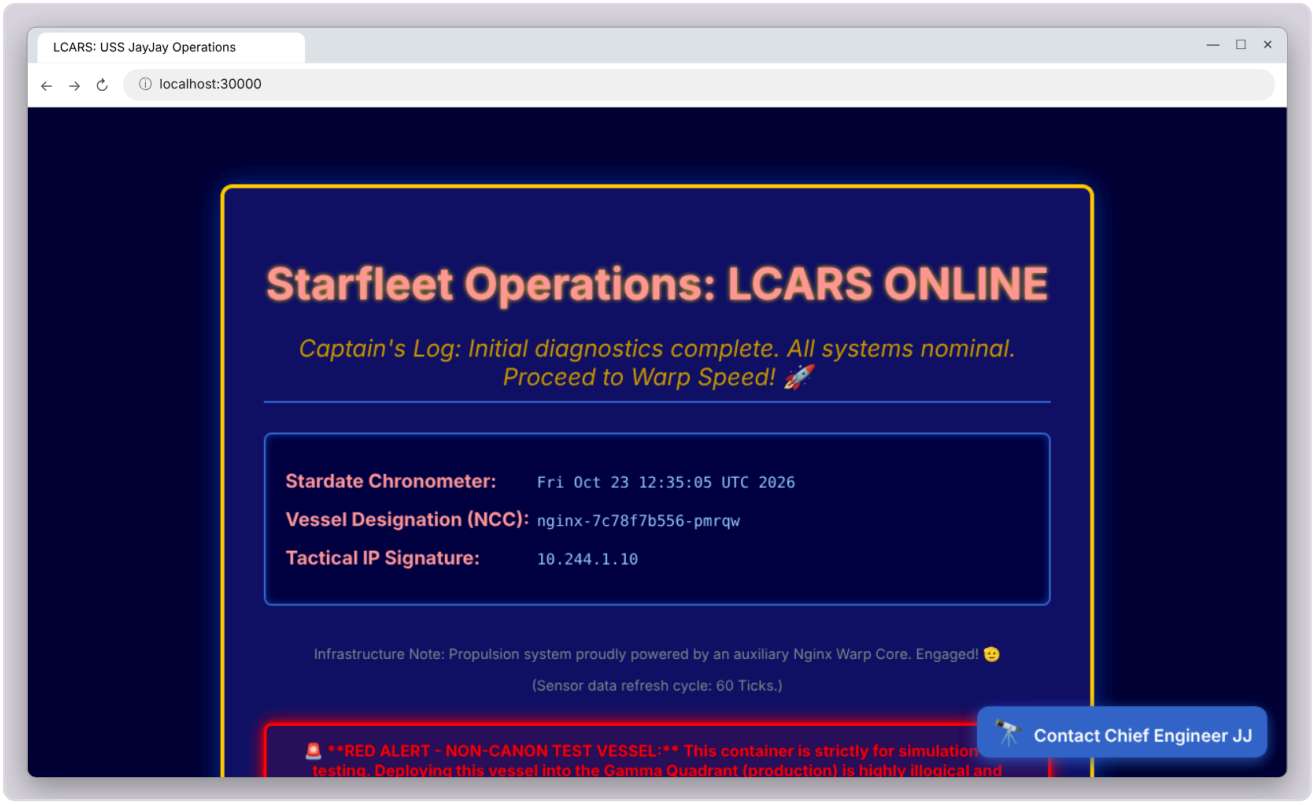
Command	Purpose
`kubectl set image deployment/nginx nginx=nginx:this-tag-does-not-exist`	Starts a rollout to an image that cannot be pulled.
`kubectl get pods`	Run about 20 seconds later to see the state.
`curl -s http://localhost:30000 | head -n 1`	Checks whether users still get a page.



Terminal 1 · captured 2026-10-23 12:35:47 UTC

Observation: The new Pod nginx-ff5d8f865-52tv2 is 0/1 ImagePullBackOff, while the 3 old Pods are still Running and serving (curl returns <!DOCTYPE html>). The rollout stopped instead of removing working Pods.

Figure 39. 6.1 Browser during the failed update

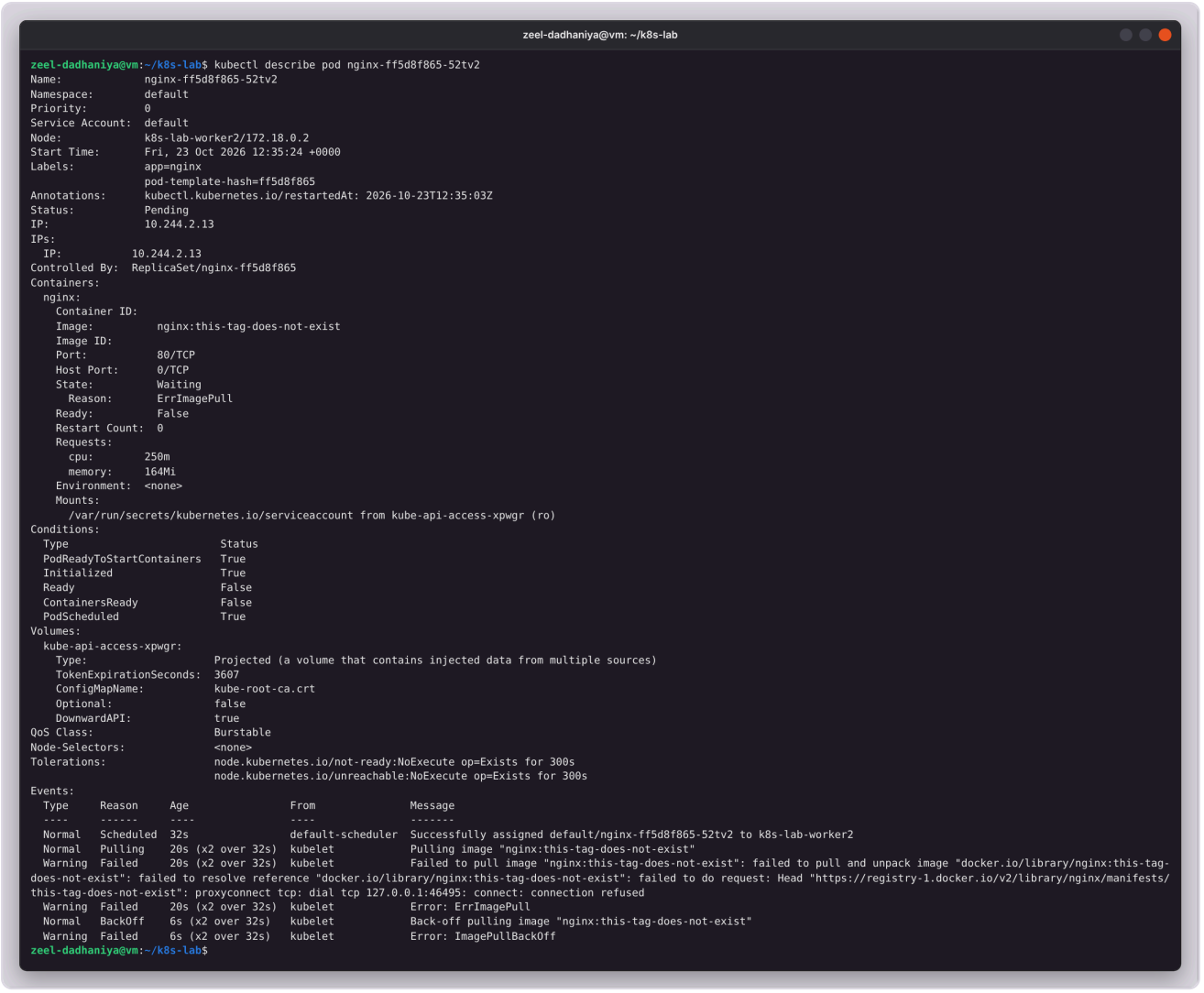


Browser · captured 2026-10-23 12:35:54 UTC

> **Observation:** A failed deployment is not automatically an outage; the site is fine.

Figure 40. 6.2 Finding the cause with kubectl describe

Command	Purpose
`kubectl describe pod nginx-ff5d8f865-52tv2`	Shows the full Pod description; the Events at the bottom record what happened.

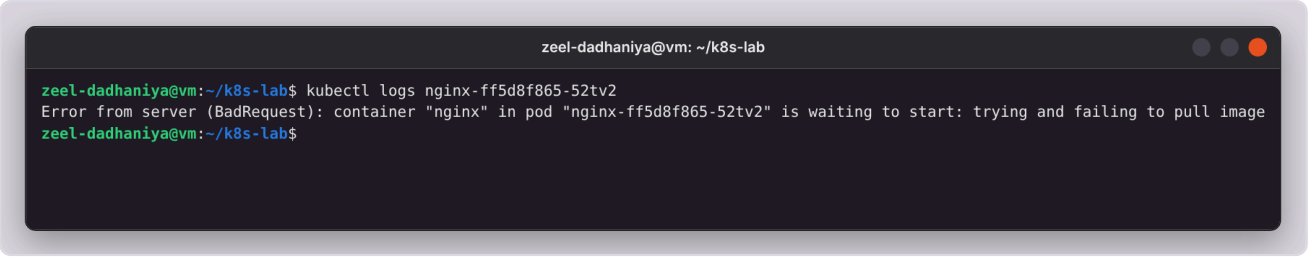


Terminal 1 · captured 2026-10-23 12:35:56 UTC

> **Observation:** Scheduled worked (assigned to k8s-lab-worker2), then the kubelet failed at Pulling: ErrImagePull, then ImagePullBackOff. On this VM the nodes have no direct internet route, so the message is a connection error. On a laptop it reads "not found". The next screenshot confirms the real cause.

Figure 41. 6.2 Why kubectl logs does not help here

Command	Purpose
`kubectl logs nginx-ff5d8f865-52tv2`	Tries to read the container's output.



Terminal 1 · captured 2026-10-23 12:36:08 UTC

Observation: The output is: container "nginx" in pod "nginx-ff5d8f865-52tv2" is waiting to start: trying and failing to pull image. There are no logs because the container never started. Use describe when it never started, and logs when it started but behaves wrongly.

Figure 42. 6.2 Extra check: does the tag exist?

Command	Purpose
`curl -s https://hub.docker.com/v2/repositories/library/nginx/tags/this-tag-does-not-exist`	Asks the Docker Hub registry API about the tag.
`curl -s -o /dev/null -w "... HTTP %{http_code}" .../tags/this-tag-does-not-exist and .../tags/stable`	Compares the HTTP status of the bad tag and of a real tag.

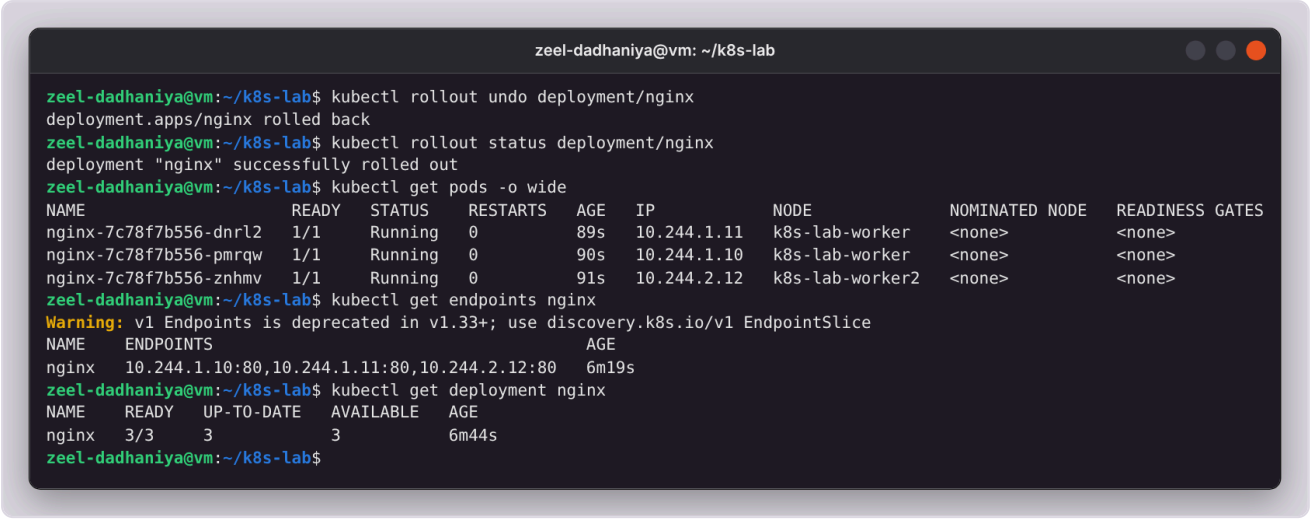


Terminal 1 · captured 2026-10-23 12:36:25 UTC

Observation: Docker Hub answers "tag 'this-tag-does-not-exist' not found" (HTTP 404), while nginx:stable returns HTTP 200. The image tag really does not exist.

Figure 43. 6.3 Fixing it with rollout undo

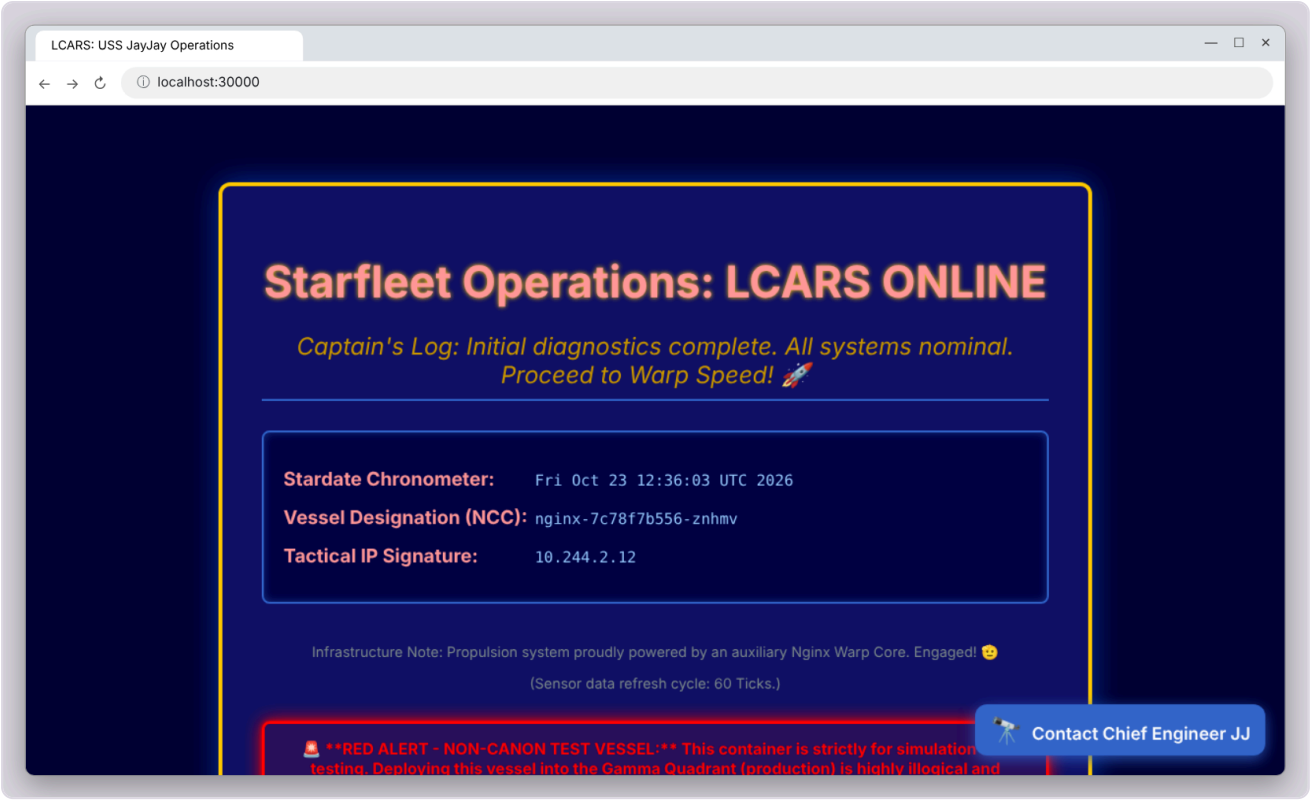
Command	Purpose
`kubectl rollout undo deployment/nginx`	Returns to the last working revision.
`kubectl rollout status deployment/nginx`	Confirms the rollout is healthy.
`kubectl get pods -o wide`	Shows 3 Running Pods.
`kubectl get endpoints nginx`	Shows 3 endpoints behind the Service.
`kubectl get deployment nginx`	Shows READY 3/3.



Terminal 1 · captured 2026-10-23 12:36:35 UTC

> **Observation:** The required end state is reached: 3/3 ready, three Pods Running, three endpoints.

Figure 44. 6.3 Browser after the fix



Browser · captured 2026-10-23 12:36:35 UTC

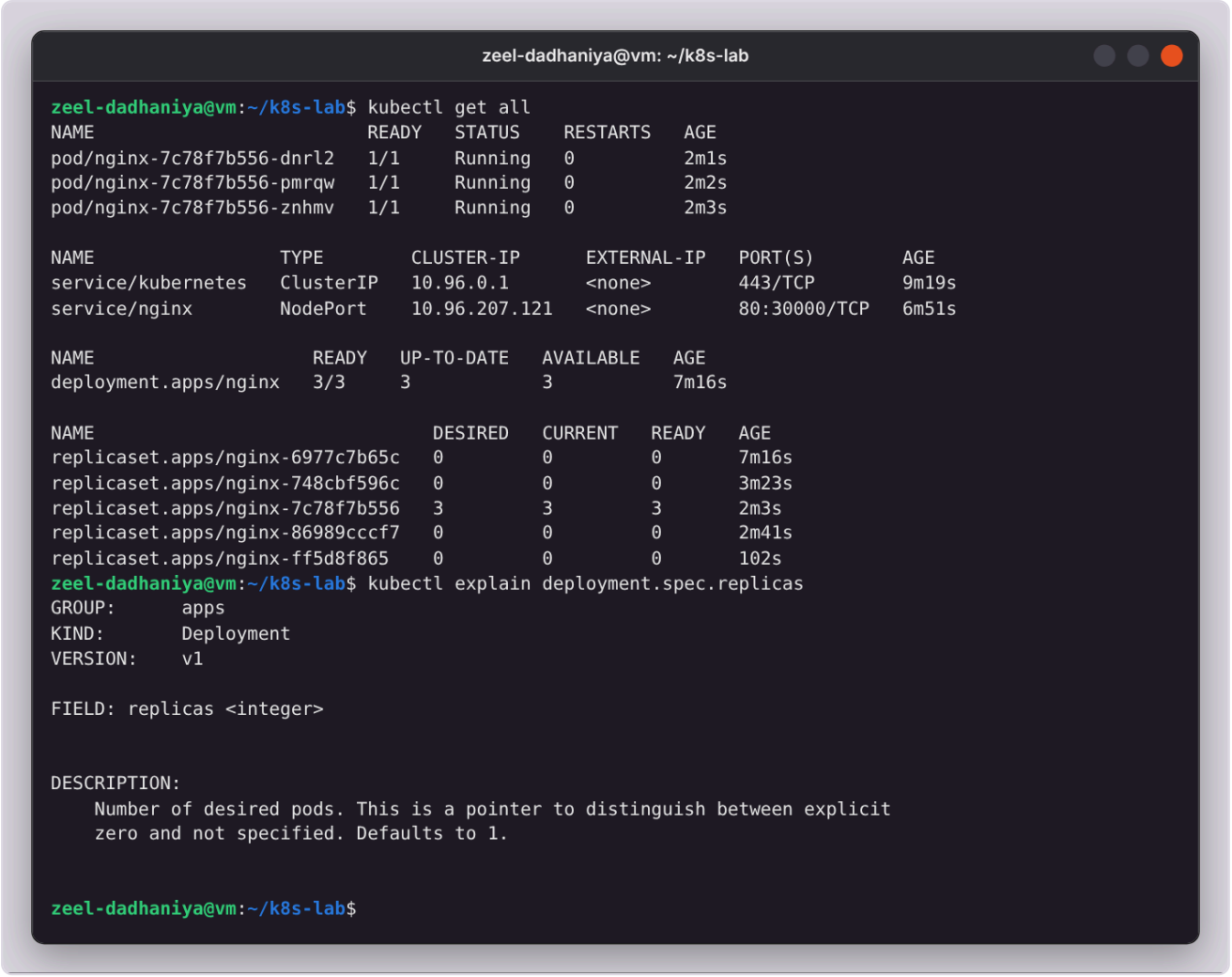
> **Observation:** http://localhost:30000 still shows the application.

Extra — Useful commands, volumes and persistent storage

The lab's command reference, plus a short exercise for the submission checklist item "volumes and persistent storage". The lab manual has no storage step, so this uses kind's built-in local-path StorageClass.

Figure 45. Commands worth keeping

Command	Purpose
`kubectl get all`	Lists every common resource in the namespace: Pods, Services, Deployments and ReplicaSets.
`kubectl explain deployment.spec.replicas`	Shows the built-in documentation for a field.

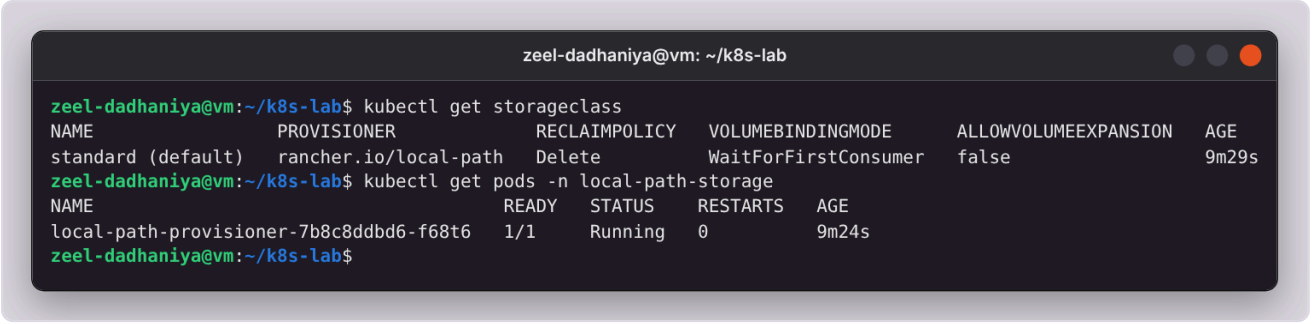


Terminal 1 · captured 2026-10-23 12:37:08 UTC

> **Observation:** kubectl get all shows the whole revision history as ReplicaSets (only one has 3 Pods). explain documents replicas as "Number of desired pods ... Defaults to 1".

Figure 46. The cluster's StorageClass

Command	Purpose
`kubectl get storageclass`	Lists the storage classes that can create volumes on demand.
`kubectl get pods -n local-path-storage`	Shows the provisioner Pod that creates the volumes.

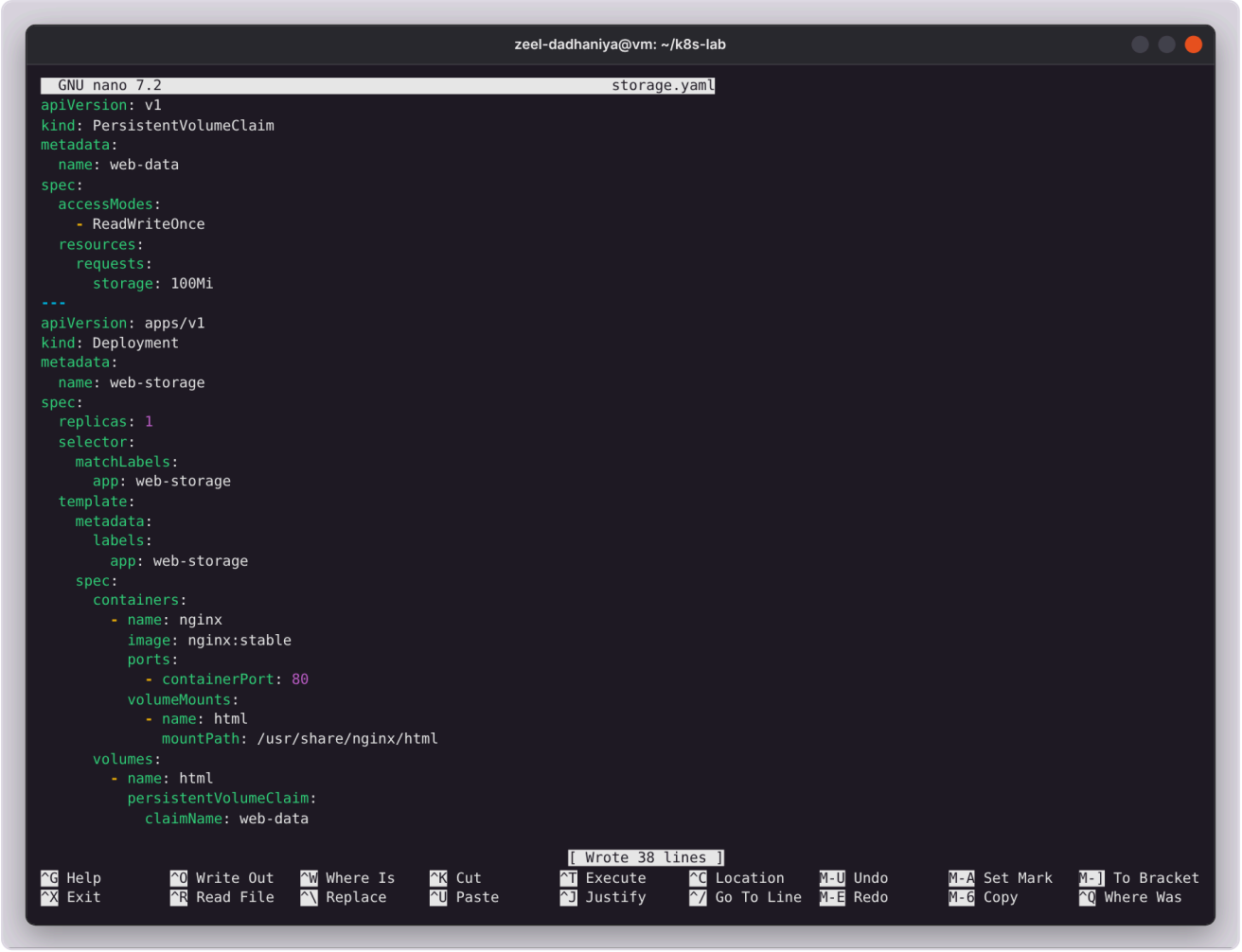


Terminal 1 · captured 2026-10-23 12:37:20 UTC

> **Observation:** standard (default) uses the rancher.io/local-path provisioner with binding mode WaitForFirstConsumer.

Figure 47. storage.yaml: a PVC and a Deployment that mounts it

Command	Purpose
`nano storage.yaml`	Writes a 100Mi ReadWriteOnce PersistentVolumeClaim (web-data) and a 1-replica nginx Deployment (web-storage) that mounts it at /usr/share/nginx/html.

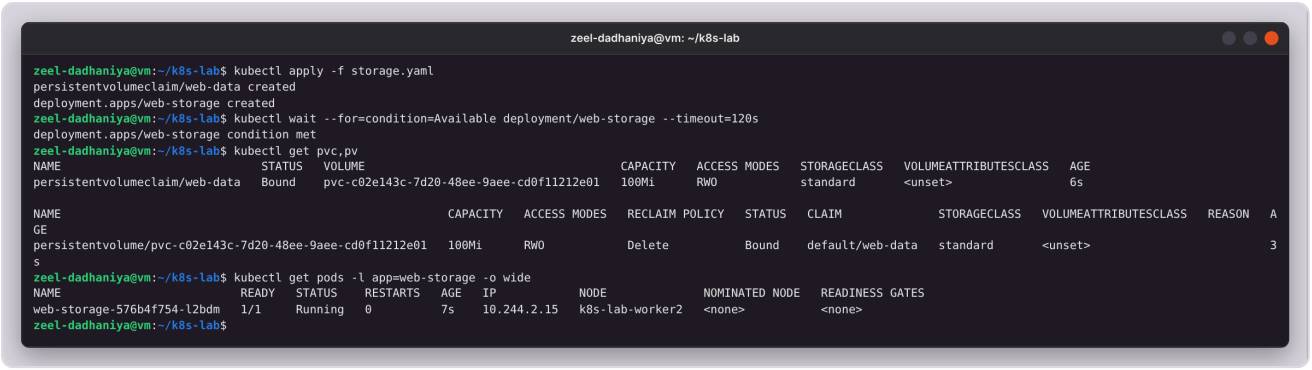


Terminal 1 · captured 2026-10-23 12:37:41 UTC

> **Observation:** The two objects are in one file, separated by ---.

Figure 48. Applying storage.yaml

Command	Purpose
`kubectl apply -f storage.yaml`	Creates the PVC and the Deployment.
`kubectl wait --for=condition=Available deployment/web-storage --timeout=120s`	Waits for the Pod to be ready (the volume is created when the Pod is scheduled).
`kubectl get pvc,pv`	Shows the claim and the volume created for it.
`kubectl get pods -l app=web-storage -o wide`	Shows the Pod using the volume.

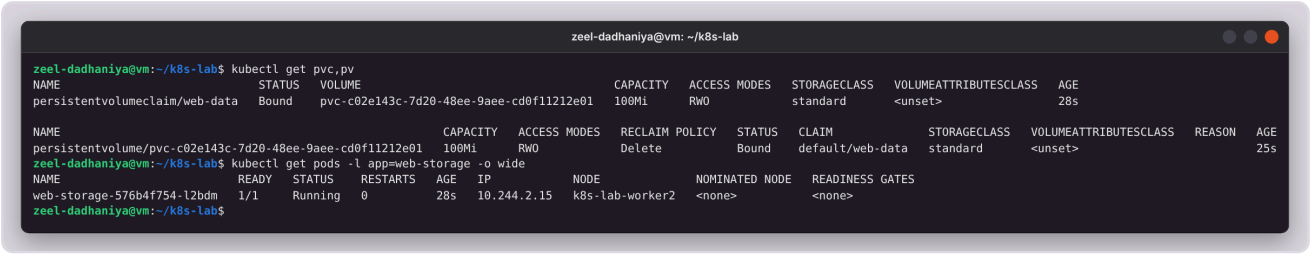


Terminal 1 · captured 2026-10-23 12:37:57 UTC

> **Observation:** Both objects were created and the Deployment became Available.

Figure 49. PVC Bound to a PersistentVolume

Command	Purpose
`kubectl get pvc,pv (wider terminal)`	Same check, in a wider window so the columns do not wrap.

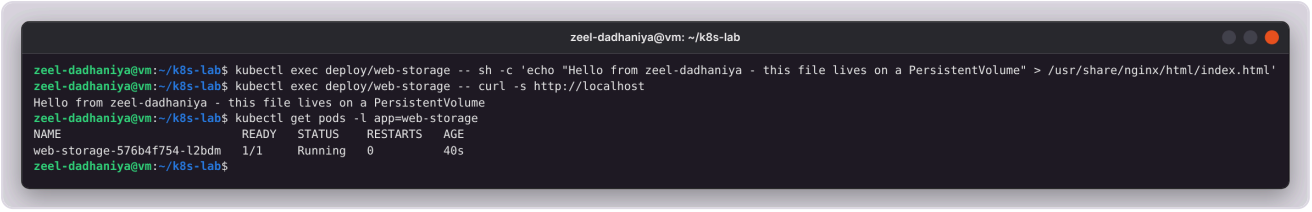


Terminal 1 · captured 2026-10-23 12:38:19 UTC

> **Observation:** web-data is Bound to pvc-c02e143c-... (100Mi, RWO, StorageClass standard). The PV was created automatically, which is dynamic provisioning.

Figure 50. Writing data into the volume

Command	Purpose
`kubectl exec deploy/web-storage -- sh -c 'echo "Hello from zeel-dadhaniya ..." > /usr/share/nginx/html/index.html'`	Runs a shell inside the Pod and writes a new index.html onto the mounted volume.
`kubectl exec deploy/web-storage -- curl -s http://localhost`	Asks nginx inside the Pod for the page.

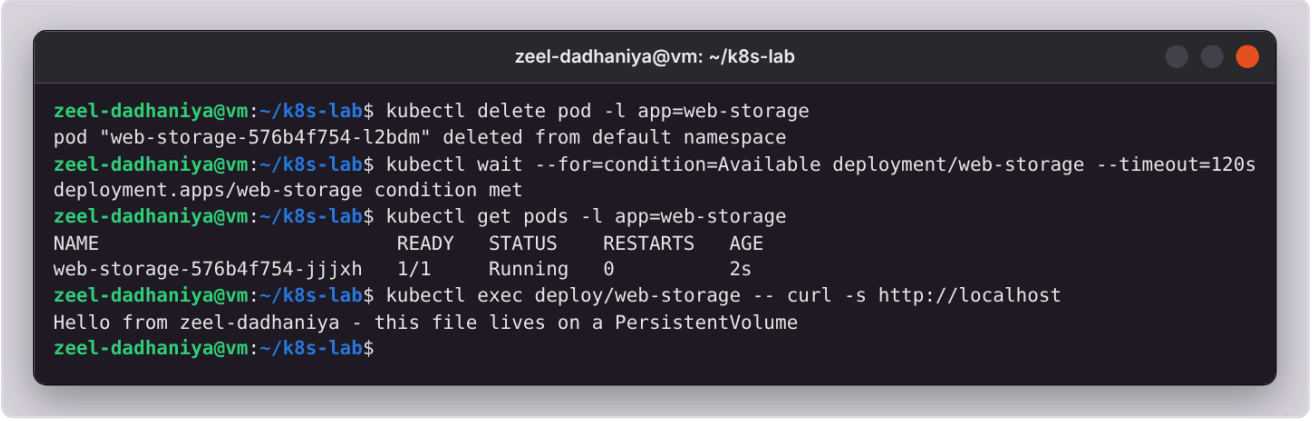


Terminal 1 · captured 2026-10-23 12:38:30 UTC

> **Observation:** nginx serves the text that was just written.

Figure 51. The data survives deleting the Pod

Command	Purpose
`kubectl delete pod -l app=web-storage`	Deletes the Pod (and with it the container's own filesystem).
`kubectl wait --for=condition=Available deployment/web-storage --timeout=120s`	Waits for the replacement Pod.
`kubectl get pods -l app=web-storage`	Shows the new Pod name (...-jjjxh instead of ...-l2bdm).
`kubectl exec deploy/web-storage -- curl -s http://localhost`	Reads the page from the new Pod.



Terminal 1 · captured 2026-10-23 12:38:34 UTC

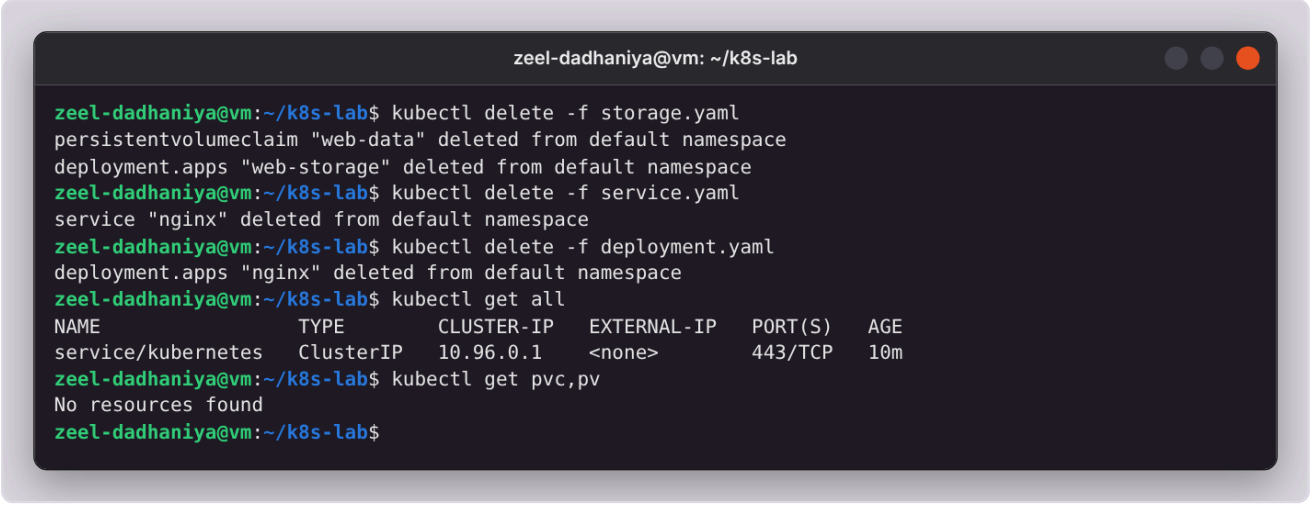
> **Observation:** The new Pod serves the same text. The file lives on the PersistentVolume, not in the disposable container, so it outlived the Pod.

Cleanup

Option A removes the apps but keeps the cluster; Option B deletes the whole cluster. Both were done, A first.

Figure 52. Option A: keep the cluster, remove the apps

Command	Purpose
`kubectl delete -f storage.yaml`	Deletes the storage Deployment and the PVC (the PV is removed automatically because its reclaim policy is Delete).
`kubectl delete -f service.yaml`	Deletes the Service.
`kubectl delete -f deployment.yaml`	Deletes the Deployment; its ReplicaSets and Pods are removed with it.
`kubectl get all`	Confirms what is left.
`kubectl get pvc,pv`	Confirms the storage is gone.

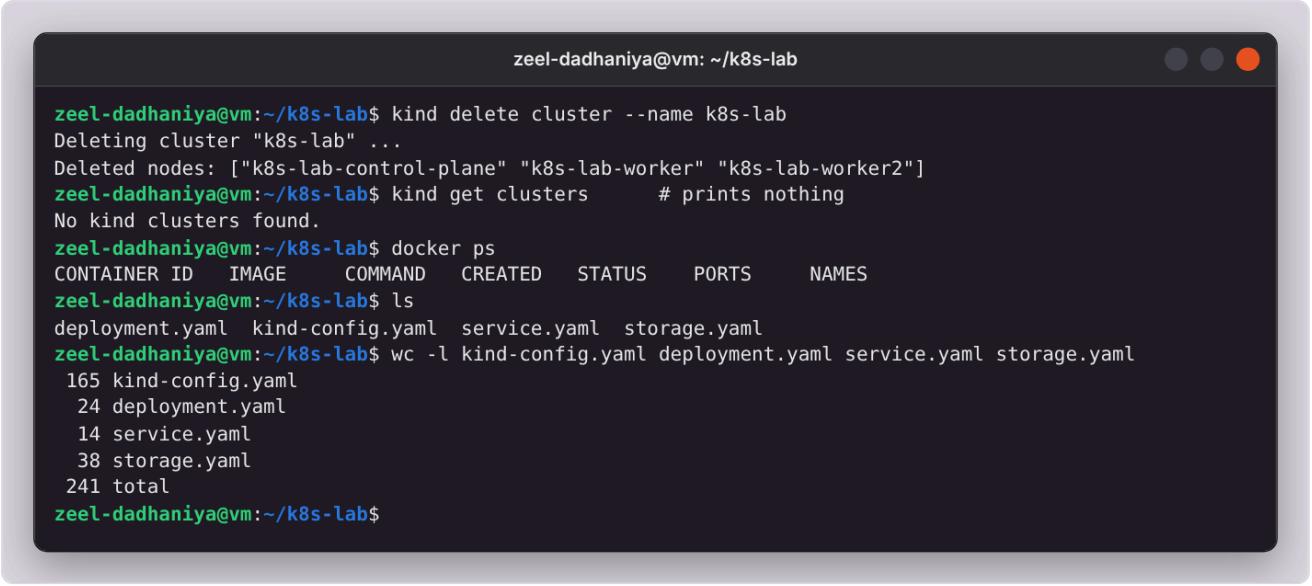


Terminal 1 · captured 2026-10-23 12:38:47 UTC

> **Observation:** Only the built-in kubernetes Service remains. No Pod or ReplicaSet was deleted by hand; deleting the owner removed everything it owned.

Figure 53. Option B: delete the whole cluster

Command	Purpose
`kind delete cluster --name k8s-lab`	Deletes the three node containers, and everything inside them.
`kind get clusters`	Lists kind clusters (none left).
`docker ps`	Confirms no containers are running.
`ls / wc -l *.yaml`	Shows that the lab files are still on disk.



Terminal 1 · captured 2026-10-23 12:39:04 UTC

> **Observation:** The cluster is gone, but kind-config.yaml, deployment.yaml, service.yaml and storage.yaml remain, so the whole lab can be rebuilt from files.

What I learned

1. Kubernetes is a **thermostat for software**. I wrote down a desired state (3 copies of nginx) and controllers kept it true, for example by replacing a deleted Pod within about 2 seconds.
2. My cluster has **one control plane and two workers**. The NoSchedule *taint* keeps application Pods off the control plane, so a bad app cannot stop me fixing things.
3. One file produced a **Deployment, a ReplicaSet and three Pods**, placed on two machines I never named. `created` means "saved in etcd", not "running", which is why I used `kubectl wait`.
4. A **Service** is one fixed address that finds Pods **by label**; `kubectl get endpoints` proved it matched 3 (then 5) Pods.
5. Deleting a Pod does not delete an app; only changing the desired state does. `rollout restart` redeploys with no file change: Pods are replaced, never restarted (RESTARTS stayed 0).
6. **`describe` when it never started, `logs` when it started but is wrong.** A failed update (ImagePullBackOff) was not an outage, and `rollout undo` fixed it.
7. A **PersistentVolumeClaim** gives a Pod storage that outlives it: data written by one Pod was still there in its replacement.

Check yourself: answers

Q1 — You applied a Deployment with replicas: 3. Which objects exist now, and who made them?
A Deployment (written by me), one ReplicaSet (made by the Deployment controller) and three Pods (made by the ReplicaSet controller). Figure for Step 2 shows nginx → nginx-6977c7b65c → nginx-6977c7b65c-xxxxx.

Q2 — You deleted a Pod. Thirty seconds later there are still three. Why?
The desired state is still 3. The ReplicaSet controller saw 2 and created a new Pod (7gn85) with a new name and IP. To really remove the app you delete the Deployment.

Q3 — The Service says app: nginx-web but the Pods say app: nginx. What does the user see?
A connection error or timeout. The Service has zero endpoints (kubectl get endpoints shows <none>), even though every Pod is Running. The failure is silent.

Q4 — After kubectl rollout restart the Pod names changed but RESTARTS is still 0. Is that correct?
Yes. The old Pods were deleted and new ones created one at a time (hash 86989cccf7 → 7c78f7b556). A restart in place would keep the name and set RESTARTS to 1.

Q5 — A new Pod is in ImagePullBackOff. Why do users still get a working site, and how do you recover?
The rolling update waits for new Pods to become ready before removing old ones, and the Service never adds the broken Pod. The 3 old Pods kept serving. Recover with kubectl rollout undo deployment/nginx.

Appendix A: all commands in order

Every command from the screenshots above, in the order it was run, with its purpose.

#	Part	Command	Purpose
1	Setup	`docker ps`	Confirms the Docker daemon is running (an empty table means no containers yet).
2	Setup	`kubectl version --client`	Prints the version of the Kubernetes command-line client.
3	Setup	`kind version`	Prints the version of kind (Kubernetes IN Docker), used to build the cluster.
4	Setup	`mkdir -p ~/k8s-lab && cd ~/k8s-lab`	Creates the lab folder (if it does not exist) and enters it. All lab files live here.
5	Setup	`pwd`	Prints the current directory to confirm the location.
6	Setup	`{ echo ...; for p in $(seq 30000 30050); do ... done; ... } > kind-config.yaml`	Writes the cluster description: one control-plane node with ports 30000–30050 mapped to the host, plus two worker nodes.
7	Setup	`ls / head -n 12 / tail -n 6 kind-config.yaml`	Shows the beginning and end of the generated file.
8	Setup	`grep -c containerPort kind-config.yaml`	Counts the port mappings (51 ports: 30000 to 30050).
9	Setup	`cat >> kind-config.yaml <<'EOF' ... restrict_oom_score_adj = true ... EOF`	Appends a containerdConfigPatches block. On this VM, containers may not lower their OOM score, and system Pods request -998. With this setting containerd limits the value instead of failing.
10	Setup	`tail -n 8 kind-config.yaml`	Shows the appended block.
11	Setup	`kind create cluster --config kind-config.yaml`	Creates three Docker containers that act as Kubernetes nodes, starts the control plane, installs networking (CNI) and storage, and joins the workers.
12	Setup	`kubectl wait --for=condition=Ready nodes --all --timeout=180s`	Waits until every node reports Ready.
13	Setup	`kubectl get nodes`	Lists the cluster's nodes, their roles and their versions.
14	Setup	`docker ps --format "{{.Names}}"`	Lists only the names of the running Docker containers.

15	Setup	`docker images | grep -E "nginx|webapps"`	Shows the three images the lab uses, already pulled by the host's Docker.
16	Setup	`docker save --platform linux/amd64 -o image.tar $img && kind load image-archive image.tar --name k8s-lab`	Exports each image to a tar file and imports it into every kind node, so the Pods can start without downloading.
17	Setup	`docker exec k8s-lab-worker crictl images | grep -E "nginx|webapps"`	Lists the images inside a worker node's container runtime to prove they arrived.
18	Step 1	`kubectl config current-context`	Shows which cluster kubectl is talking to. This should always be checked before deleting anything.
19	Step 1	`kubectl get nodes -o wide`	Lists the nodes with extra columns: internal IP, OS image, kernel and container runtime.
20	Step 1	`kubectl get pods`	Lists Pods in the default namespace.
21	Step 1	`kubectl describe node k8s-lab-control-plane | grep -i -A1 taints`	Shows the taints on the control-plane node, plus one line after.
22	Step 1	`kubectl get pods -n kube-system -o wide`	Lists the system Pods in the kube-system namespace and shows which node each one runs on.
23	Step 2	`nano deployment.yaml`	Opens the nano editor. The Deployment was typed with spaces only (no tabs), 2 spaces per level.
24	Step 2	`Ctrl+O, Enter, then Ctrl+X`	Writes the file to disk, then exits nano.
25	Step 2	`kubectl apply --dry-run=client -f deployment.yaml`	Validates the file locally without creating anything.
26	Step 2	`kubectl apply -f deployment.yaml`	Sends the desired state to the API server, which stores it in etcd.
27	Step 2	`kubectl wait --for=condition=Available deployment/nginx --timeout=120s`	Waits until the Deployment is really available instead of guessing.
28	Step 2	`kubectl get pods -o wide`	Shows the Pods with their IPs and the node each one runs on.
29	Step 2	`kubectl get deployments`	Shows the Deployment: the object that was written.
30	Step 2	`kubectl get replicasets`	Shows the ReplicaSet the Deployment created. It keeps exactly 3 Pods.
31	Step 2	`kubectl get pods`	Shows the Pods the ReplicaSet created.
32	Step 2	`kubectl describe deployment nginx | tail -n 8`	Shows the last lines of the description: conditions, the active ReplicaSet and the Events.
33	Step 3	`nano service.yaml`	Creates the Service file (apiVersion v1, type NodePort, selector app: nginx,

			port 80 → targetPort 80, nodePort 30000).
34	Step 3	`kubectl apply -f service.yaml`	Creates the Service.
35	Step 3	`kubectl get services`	Lists Services with their cluster IP and ports.
36	Step 3	`kubectl get endpoints nginx`	Shows which Pod IPs the Service sends traffic to.
37	Step 3	`curl -s http://localhost:30000 | head -n 4`	Requests the page through the NodePort and prints the first lines of the HTML.
38	Step 3	`docker port k8s-lab-control-plane | grep 30000`	Confirms that host port 30000 is mapped into the control-plane container.
39	Step 3	`kubectl port-forward service/nginx 8080:80 (Terminal 2)`	Opens a temporary tunnel from localhost:8080 to the Service's port 80. It runs until stopped.
40	Step 3	`Ctrl+C`	Stops kubectl port-forward and closes the tunnel.
41	Step 4	`kubectl scale deployment nginx --replicas=5`	Changes the desired number of Pods from 3 to 5.
42	Step 4	`kubectl rollout status deployment/nginx`	Waits until all 5 replicas are available.
43	Step 4	`kubectl get pods -o wide`	Shows the old and new Pods.
44	Step 4	`kubectl get endpoints nginx`	Shows that the Service picked up the new Pods.
45	Step 4	`kubectl scale deployment nginx --replicas=3`	Sets the desired count back to 3.
46	Step 4	`kubectl get pods -o wide`	Run twice: right away (2 Pods shutting down) and a few seconds later.
47	Step 4	`kubectl get pods -o wide -w (Terminal 2)`	Started first in the second tab to watch changes live.
48	Step 4	`kubectl get pods`	Lists the Pod names so one can be copied.
49	Step 4	`kubectl delete pod nginx-6977c7b65c-4zgk4`	Deletes one running Pod.
50	Step 4	`kubectl get pods -o wide`	Checks the result right away.
51	Step 5	`kubectl set image deployment/nginx nginx=nginx:stable-alpine`	Changes the live Deployment's container named nginx to the image nginx:stable-alpine.
52	Step 5	`kubectl rollout status deployment/nginx`	Follows the rolling update until it finishes.
53	Step 5	`kubectl get pods`	Shows the new Pods (new ReplicaSet hash 748cbf596c).

54	Step 5	`curl -sI http://localhost:30000 | grep -i server`	Reads the Server header from the running version.
55	Step 5	`nano deployment.yaml (Ctrl+_ go to line, Ctrl+K cut line, type the new line, Ctrl+O save)`	Changes the image to burgxy/webapps:v4 and the requests to 250m CPU / 164Mi memory, as in the lab.
56	Step 5	`kubectl apply -f deployment.yaml`	Compares the file with the cluster and applies the differences ("configured").
57	Step 5	`kubectl rollout status deployment/nginx`	Waits for the rolling update to finish.
58	Step 5	`kubectl get pods -o wide`	Shows Pods from the new ReplicaSet 86989cccf7.
59	Step 5	`kubectl get replicasets`	Lists all ReplicaSets the Deployment has created.
60	Step 5	`kubectl rollout undo deployment/nginx`	Rolls back to the previous revision.
61	Step 5	`kubectl rollout status deployment/nginx`	Waits for the rollback to finish.
62	Step 5	`kubectl get deployment nginx -o jsonpath='{.spec.template.spec.containers[0].image}'; echo`	Prints only the image currently in the Deployment's template.
63	Step 5	`kubectl rollout history deployment/nginx`	Lists the recorded revisions (extra check).
64	Step 5	`kubectl get replicasets`	Shows which ReplicaSet is active again.
65	Step 5	`kubectl apply -f deployment.yaml (1st time)`	The live Deployment had been rolled back, so it no longer matched the file (webapps:v4). apply re-synced it: "configured".
66	Step 5	`kubectl rollout status deployment/nginx`	Waits for that re-sync rollout.
67	Step 5	`kubectl apply -f deployment.yaml (2nd time)`	Now the file and the cluster are the same, so apply reports "unchanged" and does nothing.
68	Step 5	`kubectl get pods -l app=nginx # before`	Records the Pod names before the restart (hash 86989cccf7).
69	Step 5	`kubectl rollout restart deployment/nginx`	Asks for every Pod to be replaced, even though nothing in the file changed.
70	Step 5	`kubectl rollout status deployment/nginx`	Follows the rolling replacement.
71	Step 5	`kubectl get pods -l app=nginx # after`	Run twice: right after, and a few seconds later once the last old Pod had gone.
72	Step 6	`kubectl set image deployment/nginx nginx=nginx:this-tag-does-not-exist`	Starts a rollout to an image that cannot be pulled.

73	Step 6	`kubectl get pods`	Run about 20 seconds later to see the state.
74	Step 6	`curl -s http://localhost:30000 | head -n 1`	Checks whether users still get a page.
75	Step 6	`kubectl describe pod nginx-ff5d8f865-52tv2`	Shows the full Pod description; the Events at the bottom record what happened.
76	Step 6	`kubectl logs nginx-ff5d8f865-52tv2`	Tries to read the container's output.
77	Step 6	`curl -s https://hub.docker.com/v2/repositories/library/nginx/tags/this-tag-does-not-exist`	Asks the Docker Hub registry API about the tag.
78	Step 6	`curl -s -o /dev/null -w "... HTTP %{http_code}" .../tags/this-tag-does-not-exist and .../tags/stable`	Compares the HTTP status of the bad tag and of a real tag.
79	Step 6	`kubectl rollout undo deployment/nginx`	Returns to the last working revision.
80	Step 6	`kubectl rollout status deployment/nginx`	Confirms the rollout is healthy.
81	Step 6	`kubectl get pods -o wide`	Shows 3 Running Pods.
82	Step 6	`kubectl get endpoints nginx`	Shows 3 endpoints behind the Service.
83	Step 6	`kubectl get deployment nginx`	Shows READY 3/3.
84	Extra	`kubectl get all`	Lists every common resource in the namespace: Pods, Services, Deployments and ReplicaSets.
85	Extra	`kubectl explain deployment.spec.replicas`	Shows the built-in documentation for a field.
86	Extra	`kubectl get storageclass`	Lists the storage classes that can create volumes on demand.
87	Extra	`kubectl get pods -n local-path-storage`	Shows the provisioner Pod that creates the volumes.
88	Extra	`nano storage.yaml`	Writes a 100Mi ReadWriteOnce PersistentVolumeClaim (web-data) and a 1-replica nginx Deployment (web-storage) that mounts it at /usr/share/nginx/html.
89	Extra	`kubectl apply -f storage.yaml`	Creates the PVC and the Deployment.
90	Extra	`kubectl wait --for=condition=Available deployment/web-storage --timeout=120s`	Waits for the Pod to be ready (the volume is created when the Pod is scheduled).
91	Extra	`kubectl get pvc,pv`	Shows the claim and the volume created for it.
92	Extra	`kubectl get pods -l app=web-storage -o wide`	Shows the Pod using the volume.

93	Extra	`kubectl get pvc,pv (wider terminal)`	Same check, in a wider window so the columns do not wrap.
94	Extra	`kubectl exec deploy/web-storage -- sh -c 'echo "Hello from zeel-dadhaniya ..." > /usr/share/nginx/html/index.html'`	Runs a shell inside the Pod and writes a new index.html onto the mounted volume.
95	Extra	`kubectl exec deploy/web-storage -- curl -s http://localhost`	Asks nginx inside the Pod for the page.
96	Extra	`kubectl delete pod -l app=web-storage`	Deletes the Pod (and with it the container's own filesystem).
97	Extra	`kubectl wait --for=condition=Available deployment/web-storage --timeout=120s`	Waits for the replacement Pod.
98	Extra	`kubectl get pods -l app=web-storage`	Shows the new Pod name (...-jjjxh instead of ...-l2bdm).
99	Extra	`kubectl exec deploy/web-storage -- curl -s http://localhost`	Reads the page from the new Pod.
100	Cleanup	`kubectl delete -f storage.yaml`	Deletes the storage Deployment and the PVC (the PV is removed automatically because its reclaim policy is Delete).
101	Cleanup	`kubectl delete -f service.yaml`	Deletes the Service.
102	Cleanup	`kubectl delete -f deployment.yaml`	Deletes the Deployment; its ReplicaSets and Pods are removed with it.
103	Cleanup	`kubectl get all`	Confirms what is left.
104	Cleanup	`kubectl get pvc,pv`	Confirms the storage is gone.
105	Cleanup	`kind delete cluster --name k8s-lab`	Deletes the three node containers, and everything inside them.
106	Cleanup	`kind get clusters`	Lists kind clusters (none left).
107	Cleanup	`docker ps`	Confirms no containers are running.
108	Cleanup	`ls / wc -l *.yaml`	Shows that the lab files are still on disk.

Appendix B: YAML files

kind-config.yaml

Generated by the lab's script (165 lines). The middle port entries are left out here; the full file is in `yaml/kind-config.yaml`.

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
name: k8s-lab
nodes:
  - role: control-plane
    extraPortMappings:
      - containerPort: 30000
        hostPort: 30000
        protocol: TCP
      - containerPort: 30001
        hostPort: 30001
        protocol: TCP
      ...   (30002 – 30049, same 3 lines each)
        hostPort: 30049
        protocol: TCP
      - containerPort: 30050
        hostPort: 30050
        protocol: TCP
  - role: worker
  - role: worker
containerdConfigPatches:
  - |-
    [plugins."io.containerd.grpc.v1.cri"]
      restrict_oom_score_adj = true
```

deployment-initial.yaml

deployment.yaml as first typed in Step 2 (nginx:stable).

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:stable
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: "50m"
              memory: "64Mi"
```

deployment.yaml

deployment.yaml after the declarative edit in Step 5.1 (burgxy/webapps:v4).

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: burgxy/webapps:v4
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: "250m"
              memory: "164Mi"
```

service.yaml

NodePort Service from Step 3.

```
apiVersion: v1
kind: Service
metadata:
  name: nginx
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80          # the Service's port inside the cluster
      targetPort: 80    # the port inside the container
      nodePort: 30000   # the port you type in your browser
```

storage.yaml

PersistentVolumeClaim + Deployment from the storage exercise.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: web-data
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 100Mi
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-storage
spec:
  replicas: 1
  selector:
    matchLabels:
      app: web-storage
  template:
    metadata:
      labels:
        app: web-storage
    spec:
      containers:
        - name: nginx
          image: nginx:stable
          ports:
            - containerPort: 80
          volumeMounts:
            - name: html
              mountPath: /usr/share/nginx/html
      volumes:
        - name: html
          persistentVolumeClaim:
            claimName: web-data
```